Fakultas Rekayasa Industri
Telkom University

ENTERPRISE APPLICATION DEVELOPMENT

APSI

ANALISIS DAN PERANCANGAN SISTEM INFORMASI



MODUL 2

Class Diagram and Sequence Diagram

Arranged By: **EAD Laboratory Team**

Fakultas Rekayasa Industri Telkom University

ENTERPRISE APPLICATION DEVELOPMENT

ASISTEN PRAKTIKUM



M. Rizki Aulia Wibowo



M. Iqbal Abhipraya



Rafi Akbar Firdaus



Reza Dwi Saputra



Topas Akbar



Casta Garneta



Mellafesa Rofida



Ahmad Akmal Amran



Amelia Vega



Aisyah Nur Raihandany P.



Naufal Athalino Bakti



Clarissa Tompunu



M. Habibi Budiman



Naufal Akmal Rabbani



Fiyola Nur Alamanda



Safrina Auriya Anantasya A.



Evi Nirmalasari



Anisa Hanun



M. Hafiz Nur Irawan



Alya Davina Maharani



Hana Humaira



Nisrina Nazhifah

DAFTAR ISI

DAFTAR ISI **3**
PERATURAN PRAKTIKUM **4**
Program Learning Outcomes (PLO) & Course Learning Outcomes (CLO) **7**
LEMBAR PENGESAHAN **8**
A. Class Diagram 9
1. Manfaat Class Diagram dalam Pengembangan 10
2. Komponen Utama Class Diagram 11
3. Hubungan Antar Kelas 12
B. Sequence Diagram 16
1. Definisi 16
2. Notasi Sequence Diagram 16
3. Entity-Controller-Boundary (ECB) 24
4. Pedoman Membuat Sequence Diagram 24
LANGKAH-LANGKAH PRAKTIKUM **26**
A. Cara Membuat Class Diagram 26
B. Cara Membuat Sequance Diagram 34
CONTOH **47**
A. Studi Kasus 1 (Level Mudah) 47
B. Studi Kasus 2 (Level Kompleks) 50

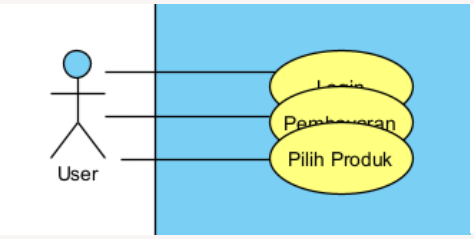
PERATURAN PRAKTIKUM

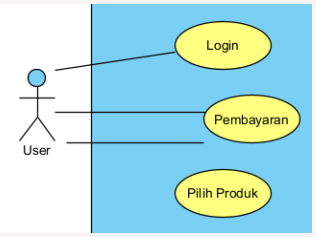
1. Setiap peserta praktikum harus datang tepat waktu sesuai dengan jadwal. Toleransi keterlambatan hadir 15 menit. Jika melebihi batas waktu tersebut diperkenankan mengikuti praktikum, namun tidak mendapatkan tambahan waktu dan/atau nilai.
2. Perizinan Praktikum :
    a. Praktikan yang ingin melakukan perizinan diwajibkan untuk menghubungi asisten praktikum group plotingan, kemudian menghubungi komisi disiplin dan mengisi form perizinan.
    b. Form perizinan dapat diakses melalui LMS atau dikirimkan oleh komisi disiplin.
    c. Izin berkaitan dengan Sakit atau Kemalangan , maka praktikan dapat memberikan surat perizinan kepada pihak Komisi Disiplin maksimal 3 hari setelah jadwal (shift) praktikum.
    d. Izin berkaitan dengan Kematian dapat memberikan surat yang ditandatangani oleh keluarga terdekat.
    e. Izin lomba atau penugasan institusi tidak berlaku apabila tidak terdapat bukti dispensasi dari Igracias. NB : screenshot dispensasi dari igracias wajib dilampirkan dan dikirim melalui form perizinan yang ada di LMS atau didapat dari komisi disiplin.
3. Seragam Praktikum :
    a. Mahasiswa wajib menggunakan celana bahan hitam (bukan chino atau jeans) pada saat praktikum.
    b. Mahasiswi wajib menggunakan rok hitam/biru gelap panjang tidak ketat pada saat praktikum
    c. Dresscode praktikum (Mengikuti Peraturan Telkom)
        - Senin : Mengenakan kemeja merah telkom atau kemeja putih polos
        - Selasa s/d Rabu : Mengenakan kemeja putih Telkom atau kemeja putih polos
        - Kamis s/d Sabtu : Mengenakan kemeja formal berkerah (bukan kerah sanghai dan bukan polo)
        - Jumat : Mengenakan kemeja/ baju batik bukan outer.
    d. Jika terdapat kendala dalam baju seragam maka praktikan diperbolehkan mengganti menggunakan kemeja putih Telkom atau kemeja putih polos.
    e. Praktikan dilarang untuk menggunakan aksesoris berlebih, seperti kalung, gelang, piercing, dan lain-lain. Kecuali aksesoris keagamaan.
    f. Membuka sepatu saat memasuki ruangan lab.
    g. Untuk pengecekan seragam bagi praktikan yang masih menggunakan topi atau jaket harap dilepas terlebih dahulu.

4. Peraturan Pengerjaan
    a. Post test dilakukan secara individu dan dilarang membuka modul.
    b. Jika praktikan ketahuan membuka modul, searching di internet, menggunakan AI, dan alat komunikasi lainnya, nilai post test akan menjadi 0.

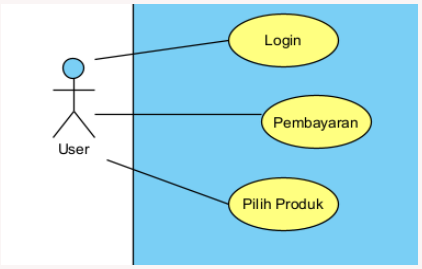
    c. Studi Kasus dikerjakan secara individu.
    d. Jawaban tidak boleh sama dengan setiap individu.
    e. Setiap diagram pada studi kasus wajib dibuat dengan memperhatikan kerapian agar mudah dipahami oleh asisten praktikum. Contoh diagram yang dianggap kurang rapi seperti notasi diagram yang bertumpuk-tumpuk, arah asosiasi yang tidak jelas, dan lain sebagainya.

(Penggambaran diagram yang kurang rapi)





(Penggambaran diagram yang rapi)



    f. Penggambaran diagram yang kurang rapi akan mendapat pengurangan nilai maksimal 7 poin dan penggambaran diagram yang rapi akan mendapat penambahan nilai maksimal sebesar 5 poin.
    g. Pengerjaan jurnal dilarang searching di internet, menggunakan AI, bertanya kepada asisten praktikum dan berdiskusi dengan praktikan lainnya, dan alat komunikasi, melakukan hal tersebut akan dikenakan plagiarisme.
    h. Submit hasil pengerjaan berupa file format PDF
    i. Format Pengumpulan Hasil Pengerjaan:
        i. File penamaan Jurnal dalam bentuk PDF:
        APSI_MODULX_KODEASISTEN_NAMA LENGKAP_NIM
        Contoh:
        APSI_ MODUL1_REDA_REZA DWI SAPUTRA_1021202200000

ii. File penamaan Tugas Pendahuluan dalam bentuk PDF:
APSI_TPX_KODEASISTEN_NAMA LENGKAP_NIM
Contoh:
APSI_ TP1_REDA_REZA DWI SAPUTRA_1021202200000

j. Screenshot hasil diagram tiap nomor dalam bentuk fullscreen dan menampilkan tanggal serta waktu. Jika tidak fullscreen maka akan dipotong 10% per nomor.

k. Salah format nama pada file pengerjaan nilai modul akan dipotong sebesar 10%
l. Jika salah mengumpulkan file nilai akan dipotong sebesar sebesar 50%
m. Terlambat mengumpulkan file pengerjaan Jurnal dibawah 2 menit saat jurnal, maka nilai jurnal akan dipotong sebesar 0% . (Jika terjadi gangguan bersama)
n. Terlambat mengumpulkan file pengerjaan Jurnal nilai jurnal akan dipotong sebesar 15% .
o. Terlambat mengumpulkan file pengerjaan Tugas Pendahuluan nilai Tugas Pendahuluan akan dipotong sebesar 35%.
p. Tidak mengumpulkan Tugas Pendahuluan maka keseluruhan modul tersebut akan dipotong 70% .
q. Segala alat komunikasi harap dimatikan dan dimasukkan kedalam tas, dan tas disimpan dibawah meja.
r. Jika ada perangkat praktikum yang bermasalah dapat menghubungi asprak yang bertugas.
s. Segala bentuk kecurangan dan plagiarisme akan diproses ke komisi disiplin dan nilai akhir modul menjadi 0.

Program Learning Outcomes (PLO) & Course Learning Outcomes (CLO)

Program Learning Outcomes (PLO)

PLO01	Mampu menganalisis permasalahan infokom yang komplek, mendefinisikan, dan memodelkan kebutuhan dalam konteks enterprise atau masyarakat dengan menerapkan ilmu dan pengetahuan dalam bidang komputasi, teknologi informasi dan komunikasi, dan disiplin lain yang relevan
PLO02	Mampu merancang, mengembangkan, mengimplementasikan, dan mengevaluasi solusi berbasis sistem informasi untuk memenuhi kebutuhan organisasi menuju data-driven organization
PLO03	Mampu untuk bekerja secara kolaboratif, proaktif, dan bertanggungjawab dalam tim untuk mencapai tujuan bersama dalam berbagai kontek profesional
PLOL08	Mampu menggunakan teknik, keahlian, atau kakas terkini yang diperlukan untuk menghasilkan solusi di bidang sistem informasi, baik dalam konteks praktikum ataupun kasus nyata

Course Learning Outcomes (CLO)

PLO01-CLO03	[PLO01-CLO03] Mampu mengidentifikasi kebutuhan sistem informasi dalam konteks enterprise atau masyarakat
PLO02-CLO01	[PLO02-CLO01] Mampu membuat perancangan sistem informasi untuk memenuhi kebutuhan organisasi menuju data-driven organization
PLO03-CLO02	[PLO03-CLO02] Mampu menerapkan sikap adaptif dalam berbagai konteks profesional untuk mencapai tujuan bersama
PLO08-CLO04	[PLO08-CLO04] Mampu menggunakan teknik, keahlian, dan perangkat dalam pengembangan aplikasi dan integrasinya dalam konteks kasus nyata

LEMBAR PENGESAHAN

Dengan lembar pengesahan ini kami menyatakan bahwa modul praktikum telah dipersiapkan dan ditinjau dengan cermat sebagai panduan dalam melaksanakan Praktikum Analisis dan Perancangan Sistem Informasi oleh Koordinator Praktikum dengan Koordinator Mata Kuliah Analisis dan Perancangan Sistem Informasi.

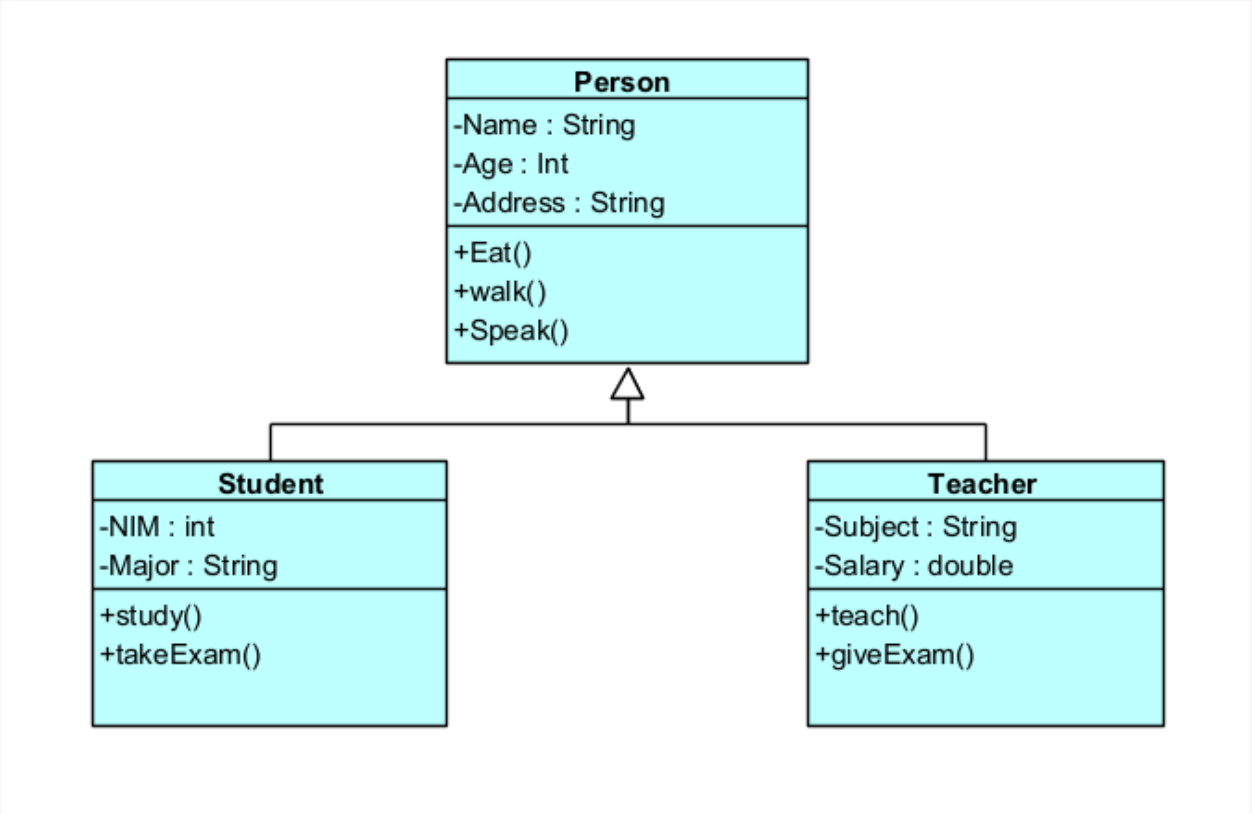
Tim Penyusun Asisten	M.Rizki Aulia Wibowo Reza Dwi Saputra
Tim Penyusun Dosen	Safara Cathasa Riverinda Rijadi, S. T., M. T.

Disetujui pada:
Bandung, 20 November 2025
Mengetahui,

Koordinator TUNC Mata Kuliah Analisis dan Perancangan Sistem Informasi	**Koordinator Praktikum Mata Kuliah Analisis dan Perancangan Sistem Informasi Tel-U Bandung**
Safara Cathasa Riverinda Rijadi, S. T., M. T NIP. 23970047	M.Rizki Aulia Wibowo NIM. 102022300208

LANDASAN TEORI

A. Class Diagram



Class diagram adalah alat visual dalam UML yang digunakan untuk menggambarkan struktur statis dari sebuah sistem perangkat lunak. Struktur statis berarti diagram ini tidak fokus pada alur atau proses (yang biasanya ditangani oleh diagram lain seperti sequence diagram), melainkan pada apa yang ada dalam sistem, yaitu kelas-kelas beserta hubungannya. Dalam konsep berorientasi objek, kelas adalah inti dari sistem karena kelas menjadi dasar untuk membuat objek atau unit nyata yang akan berjalan dalam program. Dengan class diagram, pengembang dapat merancang sistem sebelum menulis kode, sehingga mengurangi kesalahan dan mempercepat proses pengembangan.

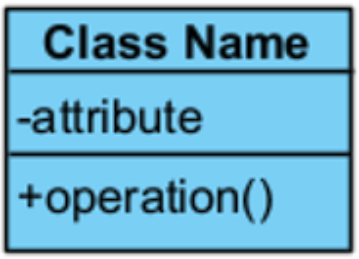
1. Manfaat Class Diagram dalam Pengembangan

a. Sebagai Blueprint Sistem: Class diagram memberi gambaran struktur sistem sebelum kode ditulis, sehingga pengembang punya acuan jelas saat implementasi.

b. Mendukung Analisis & Desain: Membantu analis dan desainer perangkat lunak memahami kebutuhan, lalu menuangkannya ke dalam rancangan kelas dan relasinya.
c. Mempermudah Komunikasi Tim: Diagram lebih mudah dipahami daripada langsung melihat kode, sehingga developer, tester, maupun stakeholder bisa memahami rancangan dengan cepat.

2. Komponen Utama Class Diagram

Class diagram terdiri dari beberapa elemen kunci yang membentuk struktur dasarnya. Berikut penjelasan masing-masing komponen:

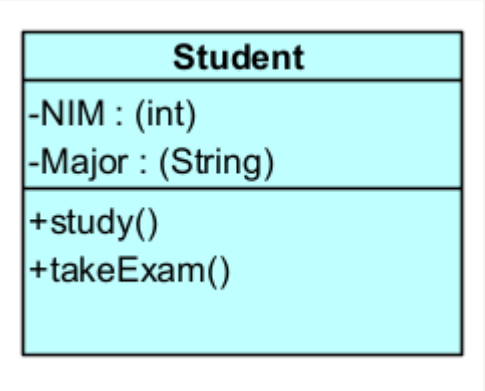


1. Kelas (Class)
    a. Definisi: Kelas adalah cetak biru (blueprint) untuk membuat objek. Kelas mendefinisikan data (atribut) dan perilaku (metode) yang dimiliki oleh
    objek yang dihasilkan darinya.
    b. Representasi: Dalam diagram, kelas digambarkan sebagai kotak persegi
    panjang yang terbagi menjadi tiga bagian:
    - Bagian atas: Nama kelas (biasanya diawali huruf kapital).
    - Bagian tengah: Daftar atribut.
    - Bagian bawah: Daftar metode.

2. Atribut (Attributes)
    a. Definisi: Atribut adalah variabel yang menyimpan data atau keadaan dari kelas. Atribut menjelaskan properti yang dimiliki oleh objek.
    b. Format: Ditulis dengan visibilitas namaAtribut: tipeData.
    c. Visibilitas:
        - (+) public: Dapat diakses dari luar kelas.
        - (-) private: Hanya dapat diakses dari dalam kelas.
        - (#) protected: Dapat diakses dari dalam kelas dan kelas turunannya.
        - ( )Default: Hanya dapat diakses didalam class atau package

d. Access Modifiers

Modifier	Class	Package	Subclass	Global
Public	✓	✓	✓	✓
Protected	✓	✓	✓	✗
Default	✓	✓	✗	✗
Private	✓	✗	✗	✗



Contoh: - NIM: Int menunjukkan atribut "NIM" bertipe Integer dan bersifat private.

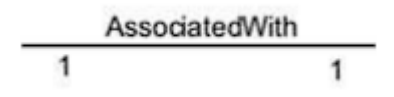
3. Methods

Method atau operation adalah fungsi atau prosedur yang dimiliki oleh suatu kelas untuk melakukan suatu tindakan atau operasi. Methods biasanya didefinisikan di dalam kelas dan dapat dipanggil oleh objek dari kelas tersebut.

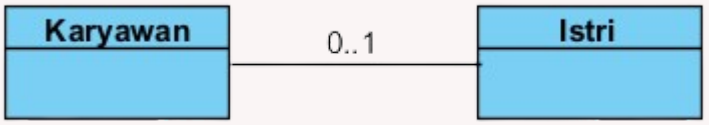
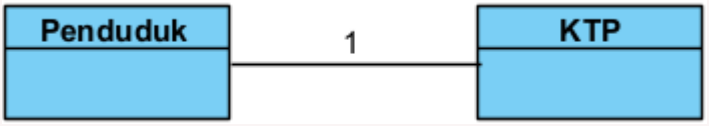
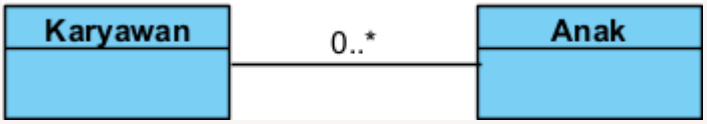
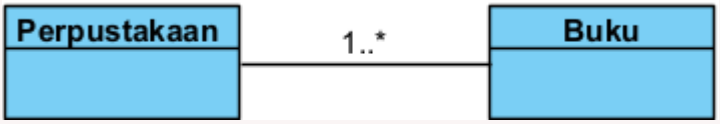
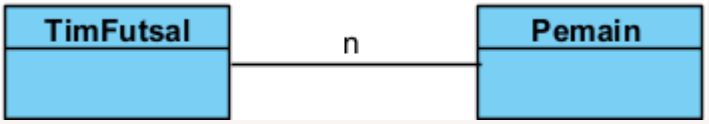
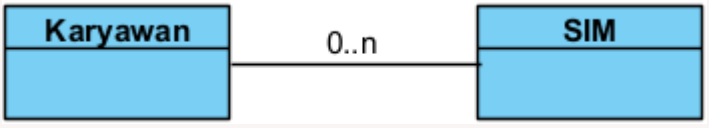
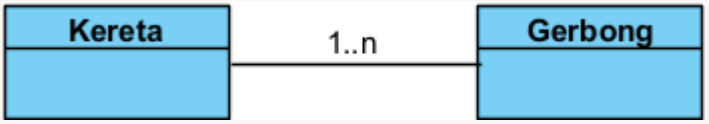
3. Hubungan Antar Kelas

Hubungan antar kelas menunjukkan bagaimana kelas-kelas saling berinteraksi atau terhubung. Berikut adalah jenis-jenis hubungan utama dalam class diagram:

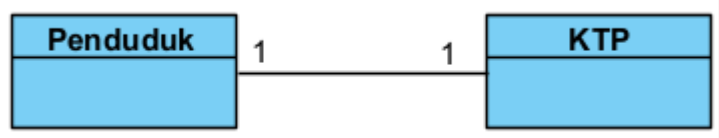
1. Asosiasi (Association)



a. Definisi:
Hubungan ini mewakili hubungan umum antar class. Hubungan ini bisa menunjukkan interaksi antar objek dari class yang berbeda. Hubungan asosiasi biasanya dilengkapi dengan simbol multiplikasi yang menunjukkan berapa banyak objek dari satu class dapat berhubungan dengan objek dari class lainnya. Berikut adalah beberapa contoh multiplikasi:

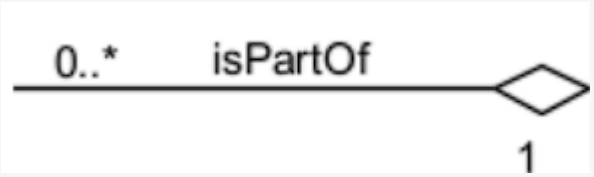
Nilai Kardinalitas	Arti	Contoh
0..1	Nol atau satu	
1	Hanya satu	
0..*	Nol atau lebih	
1..*	Satu atau lebih	
n	Hanya n (dengan n>1)	
0..n	Nol sampai n (dengan n >1)	
1..n	Satu sampai n (dengan n > 1)	

b. Representasi: Garis lurus antara dua kelas.
c. Contoh:

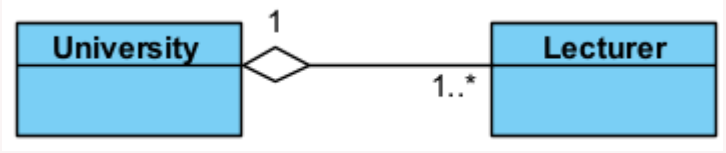


Sebagai contoh, class "Penduduk" dapat memiliki hubungan asosiasi dengan class "KTP" yang menunjukkan bahwa 1 Penduduk hanya memiliki 1 KTP dan sebaliknya.

2. Agregasi (Aggregation)

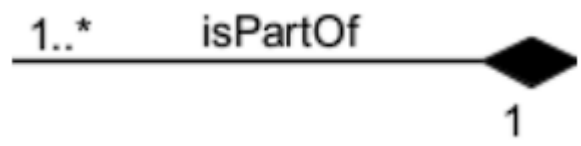


a. Definisi: Agregasi adalah hubungan antara dua kelas di mana satu kelas merupakan bagian dari kelas lainnya, tetapi tetap bisa berdiri sendiri. Dengan kata lain, meskipun suatu objek menjadi bagian dari objek lain, ia tetap bisa eksis secara mandiri tanpa bergantung sepenuhnya pada objek tersebut. Dalam class diagram, agregasi digambarkan dengan garis yang memiliki simbol berlian kosong (◊) di sisi kelas yang lebih besar atau yang memiliki bagian.
b. Representasi: Garis dengan diamond kosong di sisi kelas yang lebih besar atau memiliki bagian
c. Contoh:

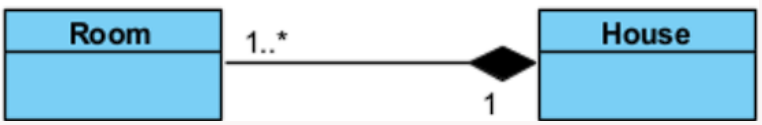


Bayangkan kelas Universitas dan kelas Dosen. Universitas memiliki banyak Dosen, tetapi Dosen tetap bisa ada meskipun tidak terdaftar di Universitas tertentu. Ini adalah contoh agregasi karena Dosen tetap bisa berdiri sendiri meskipun menjadi bagian dari Universitas.

3. Komposisi (Composition)

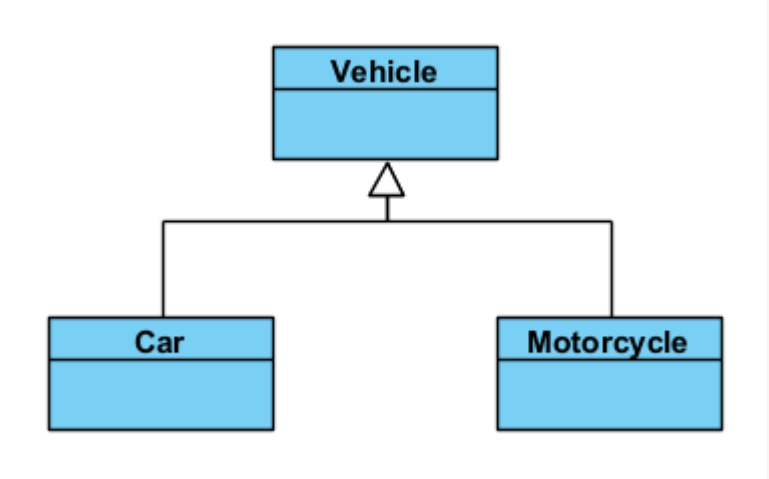


a. Definisi: Komposisi adalah hubungan yang erat antara dua kelas, di mana satu kelas tidak bisa ada tanpa yang lainnya. Jika objek keseluruhan dihapus, maka bagian-bagiannya juga ikut hilang. Dalam class diagram, komposisi digambarkan dengan garis yang memiliki simbol berlian penuh (◆) di sisi kelas yang lebih besar atau yang memiliki bagian.
b. Representasi: Garis dengan diamond terisi di sisi kelas yang memiliki bagian.
c. Contoh:



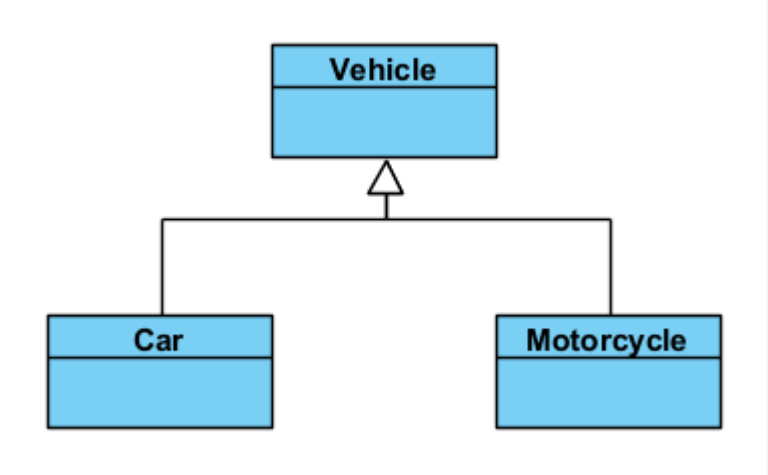
Bayangkan kelas House dan kelas Room. Rumah terdiri dari beberapa ruangan, dan jika rumah dihancurkan, maka ruangan-ruangan di dalamnya juga ikut hilang. Ruangan tidak bisa berdiri sendiri tanpa Rumah, sehingga ini adalah contoh komposisi.

4. Generalisasi (Generalization)



a. Definisi: Generalisasi adalah hubungan antara kelas induk dan kelas anak, yang menunjukkan bahwa suatu kelas adalah versi khusus dari kelas lainnya. Hubungan ini disebut juga sebagai "is-a relationship", yang berarti bahwa kelas anak mewarisi sifat dan perilaku dari kelas induknya. Dalam class diagram, generalisasi digambarkan dengan garis lurus yang memiliki panah kosong (△) yang menunjuk ke kelas induk.

b. Representasi: Garis dengan panah segitiga kosong menuju kelas induk.
c. Contoh:



Vehicle adalah kelas induk, sementara Car dan Motorcycle adalah kelas anaknya. Car dan Motorcycle mewarisi atribut dan metode dari Vehicle, seperti kecepatan, merk, dan metode untuk mengemudi.

B. Sequence Diagram

1. Definisi

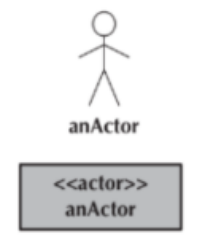
Diagram sequence adalah salah satu jenis diagram dalam UML (*Unified Modelling Language*) yang menjelaskan bagaimana objek-objek dalam sebuah sistem berinteraksi dan berkomunikasi satu sama lain. Bayangkan seperti sebuah alur cerita yang menunjukkan pertukaran pesan antar komponen sistem dari waktu ke waktu.

Diagram ini memiliki dua pendekatan utama yang berbeda. Diagram generik menggambarkan berbagai kemungkinan skenario dalam satu use case, memberikan gambaran menyeluruh tentang potensi interaksi dalam sistem dan berguna untuk memahami keseluruhan alur kerja yang mungkin terjadi. Sementara itu, diagram individual fokus pada satu skenario spesifik tertentu, mendetailkan urutan tepat dari interaksi antar objek dan membantu memahami alur kerja yang sangat konkret.

Diagram sequence paling sering digunakan pada dua tahap penting pengembangan sistem, yaitu tahap analisis dan desain. Pada tahap analisis, diagram ini membantu memahami kebutuhan dan alur sistem, sedangkan pada tahap desain, diagram digunakan untuk merancang detail interaksi antar komponen sistem. Tujuan utamanya adalah menjelaskan dengan jelas bagaimana objek-objek dalam sistem saling berkomunikasi dan bertukar pesan sesuai urutan waktu.

2. Notasi Sequence Diagram

a. Aktor



Aktor adalah orang atau sistem di luar suatu sistem yang menggunakan sistem untuk mencapai tujuannya. Aktor berinteraksi dengan sistem dengan mengirim atau menerima pesan.

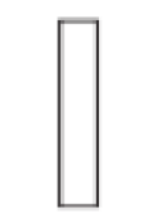
Contoh:



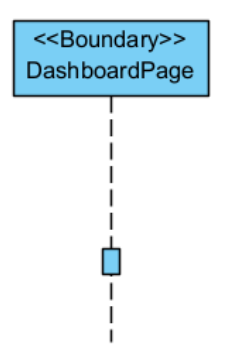
Keterangan:

Seorang Customer yang berinteraksi dengan aplikasi belanja. Dalam kasus lain, aktor bisa berupa Mahasiswa atau Dosen.

b. **Object, Lifeline, dan Execution occurance**

Nama	Notasi	Deskripsi
Object		Objek adalah bagian dari sistem yang ikut berinteraksi, baik dengan menerima atau mengirim pesan. Letaknya biasanya di bagian atas diagram.
Lifeline		Lifeline berfungsi untuk menunjukkan alur perjalanan dari sebuah objek selama skenario berlangsung.
Execution occurance		Execution occurrence adalah persegi panjang yang biasanya ditempatkan di atas lifeline. Fungsinya untuk menunjukkan bahwa suatu objek sedang dieksekusi dan berinteraksi dengan objek lain dalam sistem.

Contoh:

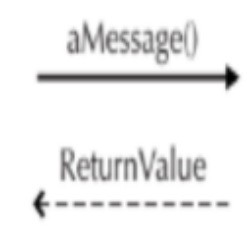


Keterangan:

Dalam contoh yang diberikan, terdapat ilustrasi sebuah objek, lifeline, dan execution occurrence yang berperan sebagai boundary,

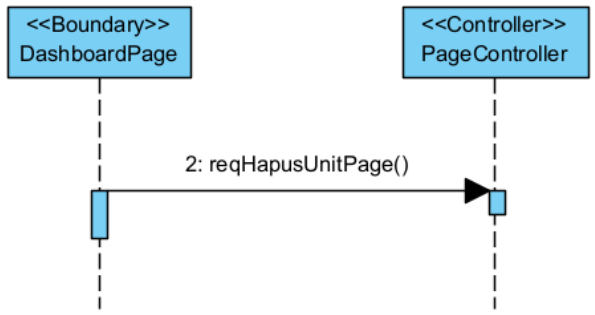
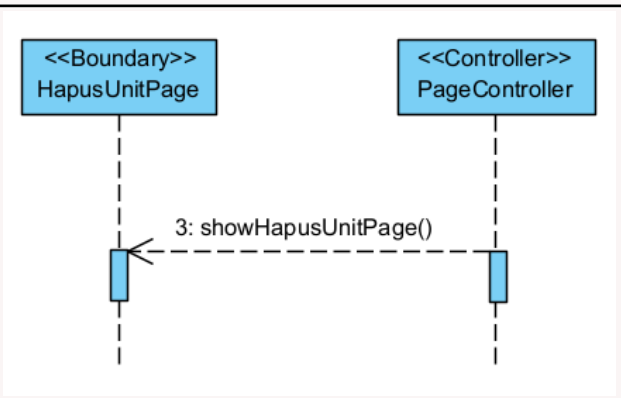
yaitu *DashboardPage*. Selain itu, contoh lain bisa berupa entity atau controller, seperti *UnitApartemen* dan *PageController*.

c. **Message**



Message digunakan untuk mengirim informasi dari satu objek ke objek lainnya. Pengiriman pesan ditandai dengan panah penuh yang disebut notasi call message, sedangkan penerimaan pesan ditandai dengan panah putus-putus yang disebut reply message.

Contoh:

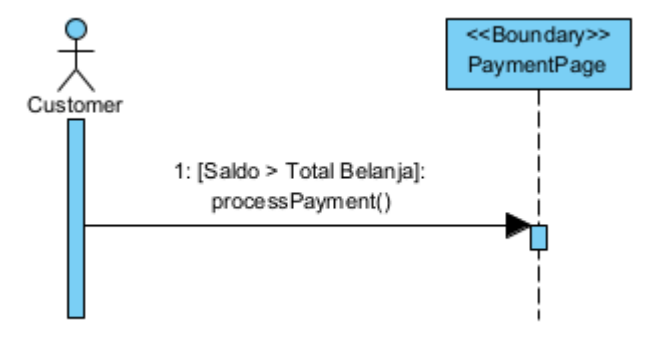
Nama	Contoh	Keterangan
Call Message		Dalam contoh yang diberikan, terdapat ilustrasi sebuah pesan (*message*). Pada gambar pertama, method *reqHapusUnitPage()* dikirim dari *:DashboardPage* ke *:PageController*. Sementara itu, pada gambar kedua, method *showHapusUnitPage()* dikirim sebagai pesan dari controller *:PageController* ke boundary *:HapusUnitPage*.
Reply Message		

d. **Guard Condition**



Guard condition adalah syarat yang harus terpenuhi agar suatu pesan bisa dikirim.

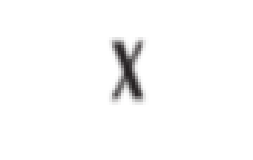
Contoh:



Keterangan:

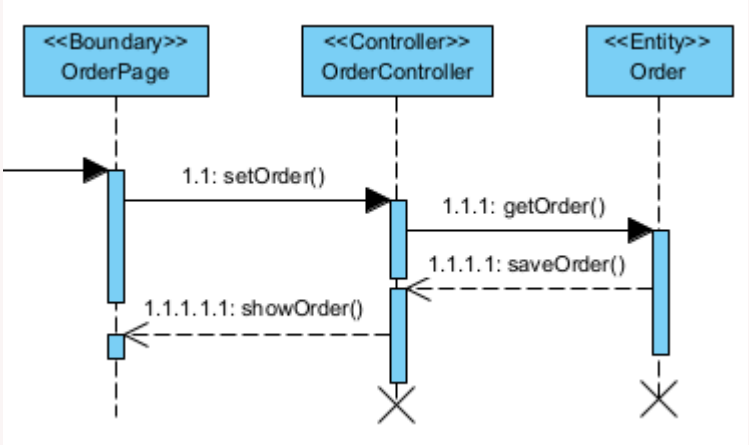
Dalam contoh ini, Customer mengirimkan permintaan pembayaran ke PaymentPage. Namun, transaksi hanya akan diproses jika syarat Saldo > TotalBelanja terpenuhi. Jika saldo tidak mencukupi, sistem tidak akan melanjutkan pembayaran.

e. **Object Destruction**



Object destruction ditandai dengan simbol "X" yang menunjukkan bahwa suatu objek keluar dari sistem. Simbol ini ditempatkan di akhir lifeline untuk menandakan bahwa objek tersebut tidak lagi berinteraksi dengan sistem. Ini juga menunjukkan *destruction occurrence*, yaitu saat objek dihapus atau dihancurkan dari sistem.

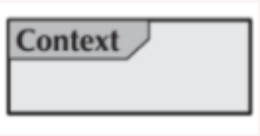
Contoh:



Keterangan:

Dalam contoh yang diberikan, terdapat dua ilustrasi object destruction, di mana lifeline *:OrderController* dan *:Order* berhenti berkomunikasi dengan sistem setelah proses pemesanan selesai. Setelah pesanan ditampilkan di *:OrderPage*, kedua objek tersebut tidak lagi aktif dalam sistem.

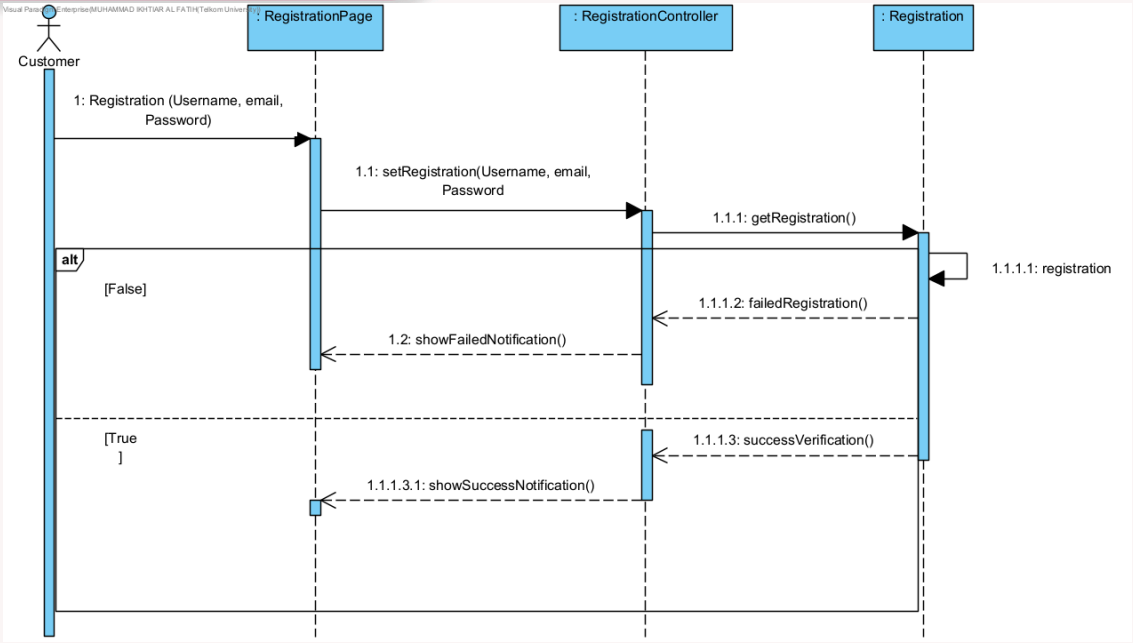
f. **Frame**



Frame digunakan untuk membungkus skenario khusus dalam sequence diagram. Ada beberapa jenis frame, misalnya:

- Alt (Alternatif)
  Menunjukkan percabangan kondisi, misalnya ketika registrasi pengguna bisa berhasil atau gagal.
- Loop (Perulangan)
  Menunjukkan proses yang berulang.

Contoh:



Keterangan:

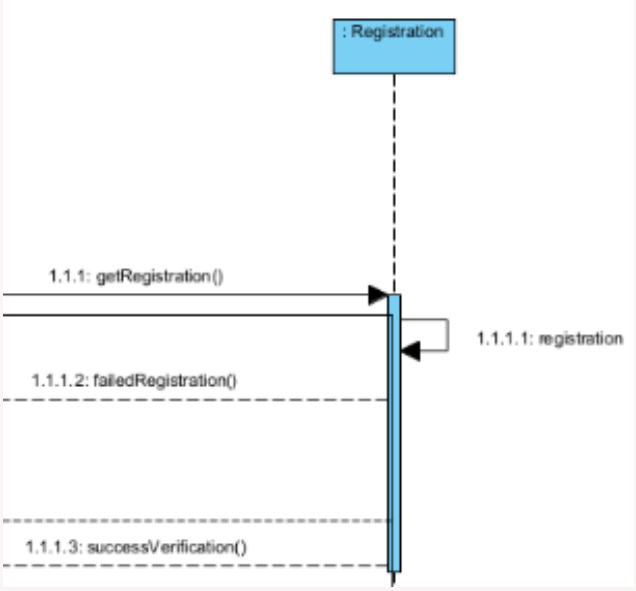
Pada contoh di atas, terdapat salah satu contoh dari frame yaitu notasi alt atau alternatif, di mana terdapat proses registrasi pengguna dalam sistem. Dalam proses ini, terdapat percabangan yang menentukan dua kemungkinan hasil, yaitu apakah registrasi berhasil atau gagal. Jika registrasi gagal, sistem akan mengirimkan pesan kegagalan dan menampilkan notifikasi kepada pengguna. Sebaliknya, jika registrasi berhasil, sistem akan mengirimkan pesan konfirmasi sukses dan menampilkan notifikasi keberhasilan. Kedua hasil ini akan menghasilkan kondisi dengan pesan yang dikirim sesuai dengan hasil verifikasi.

g. **Self Message**



Self Message adalah pesan yang dikirim oleh suatu objek kepada dirinya sendiri atau ke lifeline yang sama.

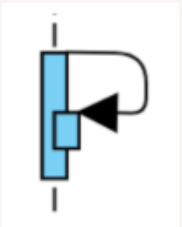
Contoh:



Keterangan:

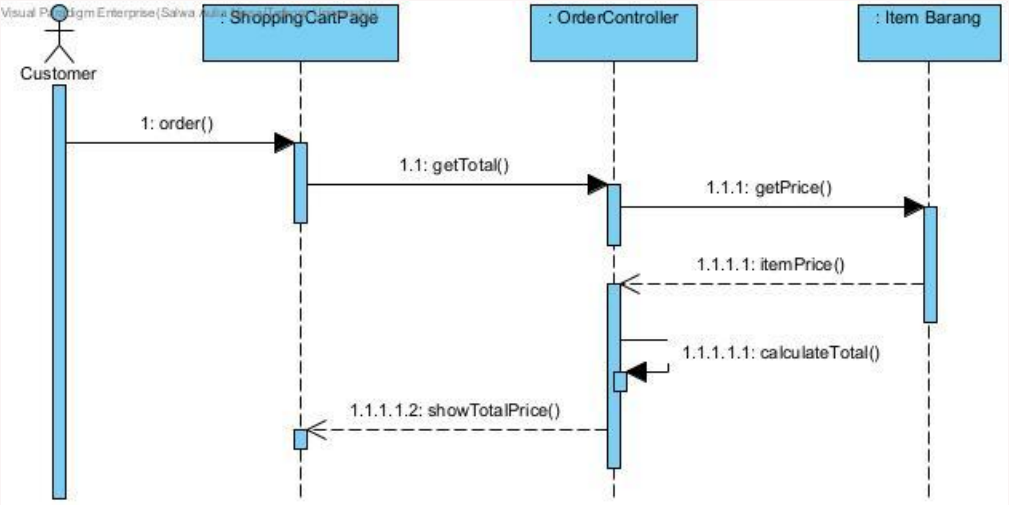
Pada gambar di atas, terdapat contoh dari self message pada lifeline : Registration, di mana terdapat pesan registration() yang menunjukkan bahwa sistem melakukan proses pendaftaran secara internal. Setelah menerima permintaan registrasi melalui pesan getRegistration(), sistem akan memvalidasi data yang diberikan sebelum mengembalikan hasil. Jika validasi gagal, pesan failedRegistration() akan dikirimkan, sedangkan jika validasi berhasil, sistem akan mengirimkan pesan successVerification() untuk melanjutkan proses registrasi ke tahap berikutnya.

h. **Recursive Message**



Recursive Message adalah pesan yang memicu pemanggilan fungsi atau metode berulang pada objek yang sama.

Contoh:



Keterangan:

Pada gambar di atas, terdapat contoh recursive message dengan pesan calculateTotal(). Dalam diagram ini, setelah mendapatkan harga barang dari entitas ItemBarang, lifeline OrderController mengirimkan pesan calculateTotal() ke dirinya sendiri. Pesan ini digunakan untuk menghitung total harga akhir dari semua item yang ada di dalam shopping cart, sehingga sistem dapat menentukan jumlah yang harus dibayarkan oleh pengguna.

3. Entity-Controller-Boundary (ECB)

Entity-Controller-Boundary (ECB) adalah pola arsitektur perangkat lunak yang digunakan dalam pengembangan aplikasi berorientasi objek. Pola ini membantu memperjelas struktur dan alur kerja sistem dalam diagram sequence dengan membagi sistem ke dalam tiga bagian utama:

1. Entity: Menyimpan data dan menangani logika bisnis.
2. Controller: Mengatur proses, mengendalikan alur kerja, dan menghubungkan entity dengan boundary.
3. Boundary: Berperan sebagai antarmuka antara sistem dan pengguna atau sistem eksternal, menerima input serta menampilkan output.

Sebagai contoh, dalam sebuah diagram sequence, setelah mendapatkan harga barang (*item price*) dari *entity* ItemBarang, *lifeline* OrderController akan mengirim pesan calculateTotal() ke dirinya sendiri untuk menghitung total harga semua item dalam keranjang belanja (*shopping cart*).

4. Pedoman Membuat Sequence Diagram

1. Identifikasi Aktor dan Objek

   Tentukan semua pihak yang Langkah pertama dalam menyusun diagram sequence adalah mengidentifikasi semua aktor dan objek yang terlibat dalam sistem. Aktor bisa berupa pengguna manusia atau sistem eksternal yang berinteraksi dengan sistem utama, sedangkan objek adalah komponen dalam sistem yang menerima dan mengirim pesan. Dalam diagram sequence, aktor dan objek disusun dari kiri ke kanan berdasarkan urutan keterlibatan aktor dalam skenario use case. Aktor atau objek yang pertama kali memulai interaksi ditempatkan di posisi paling kiri untuk mencerminkan alur interaksi yang logis dan mudah dipahami.

2. Gunakan Notasi yang Tepat

   Setelah aktor dan objek diidentifikasi, langkah berikutnya adalah menggunakan notasi yang sesuai agar diagram lebih mudah dibaca dan dipahami. Aktor biasanya digambarkan sebagai ikon manusia, sedangkan objek direpresentasikan dengan persegi panjang yang berisi nama objek tersebut. Hubungan antar aktor dan objek digambarkan dengan garis dan panah untuk menunjukkan alur komunikasi atau pesan yang dikirim. Menggunakan notasi yang standar membantu menjaga konsistensi dan memudahkan interpretasi diagram oleh berbagai pemangku kepentingan.

3. Membuat Pesan yang Jelas dan Ringkas

   Setiap pesan yang dikirim dalam diagram harus memiliki nama yang jelas dan deskriptif agar dapat menjelaskan tindakan atau informasi yang dikirim. Misalnya, daripada hanya menggunakan *"Login()"*, lebih baik menggunakan *"LoginUser()"* untuk memperjelas maksud dari pesan tersebut. Jika sebuah pesan memiliki nilai yang dikembalikan dan tidak langsung jelas, sebaiknya tambahkan return message agar lebih eksplisit, seperti *"ValidasiBerhasil()"* untuk menunjukkan bahwa validasi telah berhasil dilakukan. Nama pesan dan return message juga perlu ditempatkan di dekat ujung panah masing-masing agar lebih mudah dipahami.

4. Menyusun Pesan Berdasarkan Urutan

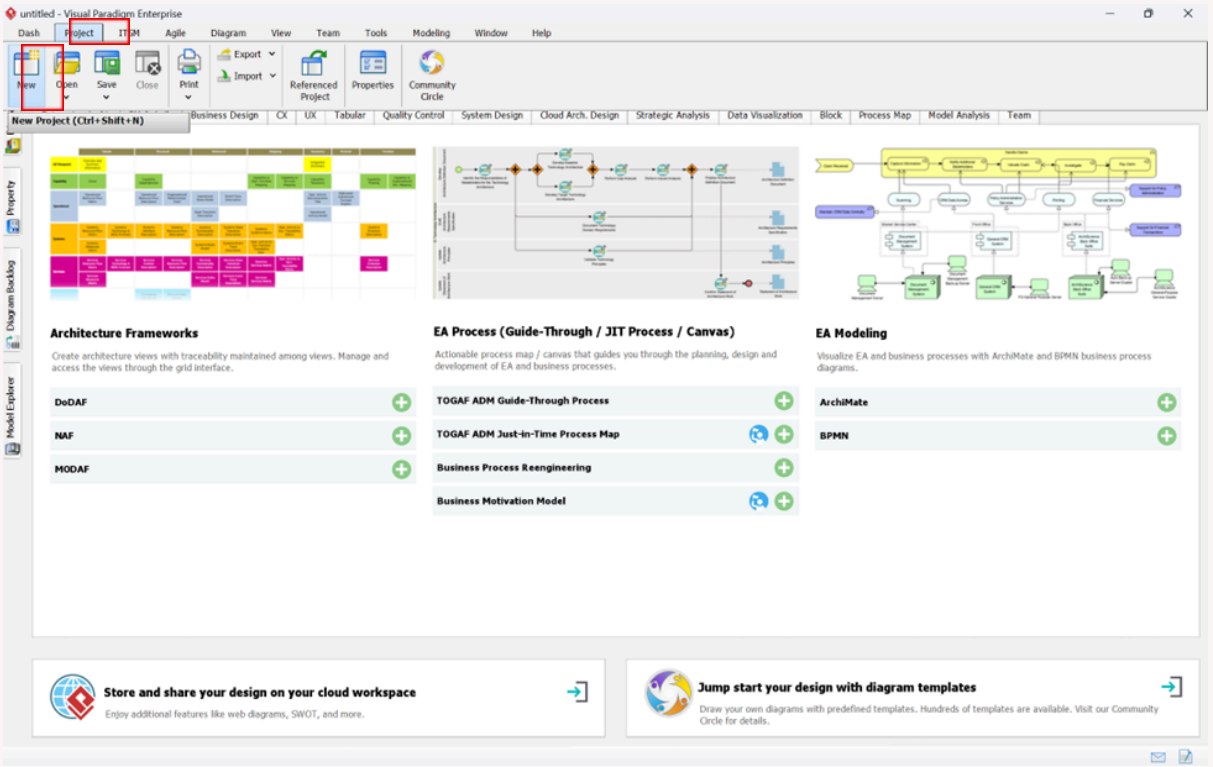
   Agar diagram lebih terstruktur dan mudah dibaca, pesan-pesan harus disusun secara berurutan dari kiri ke kanan, mengikuti kebiasaan membaca. Penyusunan ini membantu menghindari kebingungan dalam memahami alur komunikasi antar objek. Selain itu, pesan harus tetap

berada di bawah objek yang mengirim atau menerimanya untuk menjaga keteraturan diagram. Dengan mengikuti aturan ini, diagram sequence akan lebih terorganisir dan mudah dipahami oleh pengembang serta pihak terkait lainnya.

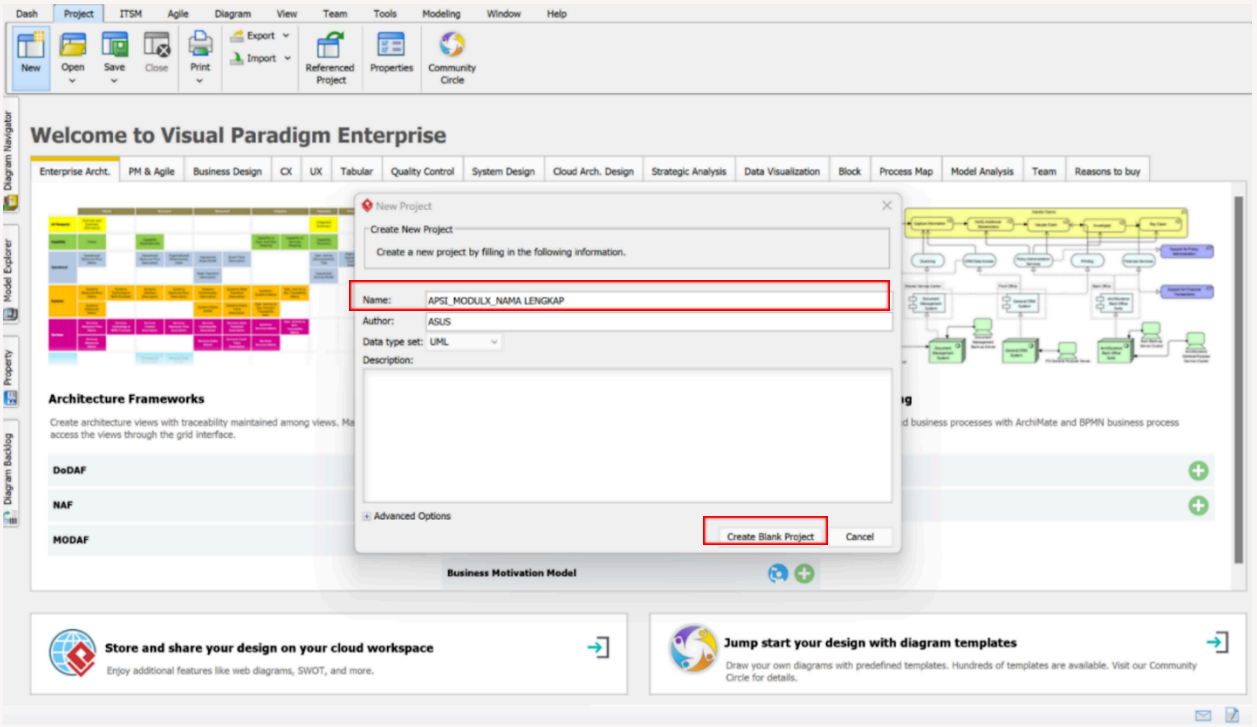
LANGKAH-LANGKAH PRAKTIKUM

A. Cara Membuat Class Diagram

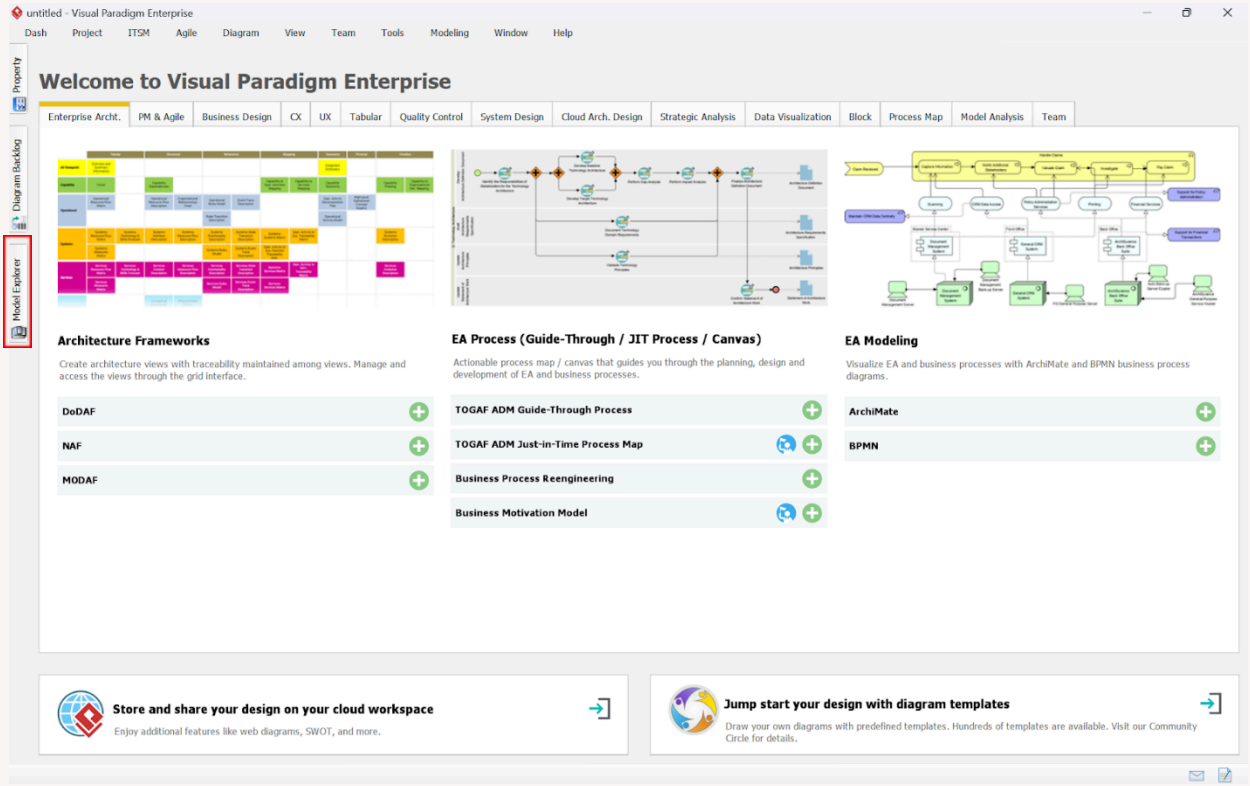
1. Membuka Visual Paradigm
2. Pada tampilan awal Visual Paradigm, klik Project > klik New



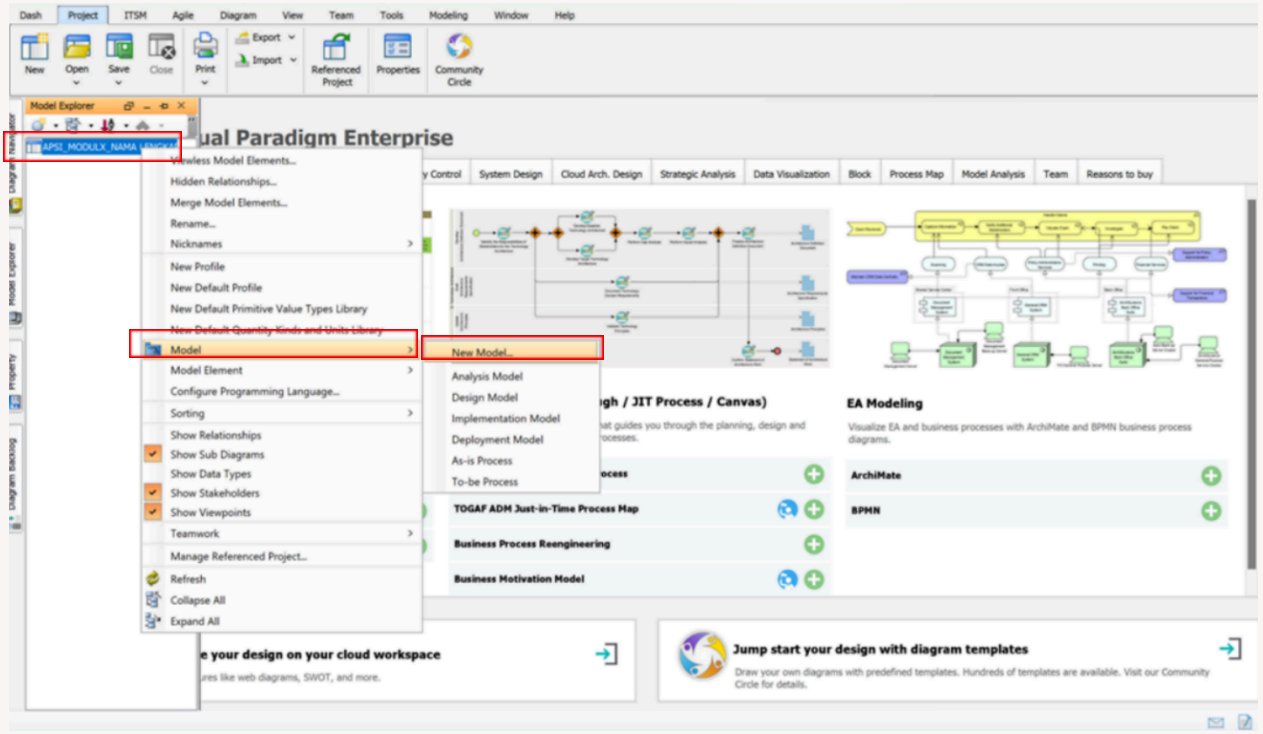
3. Isi Name sesuai dengan ketentuan , kemudian klik Create Blank Project.



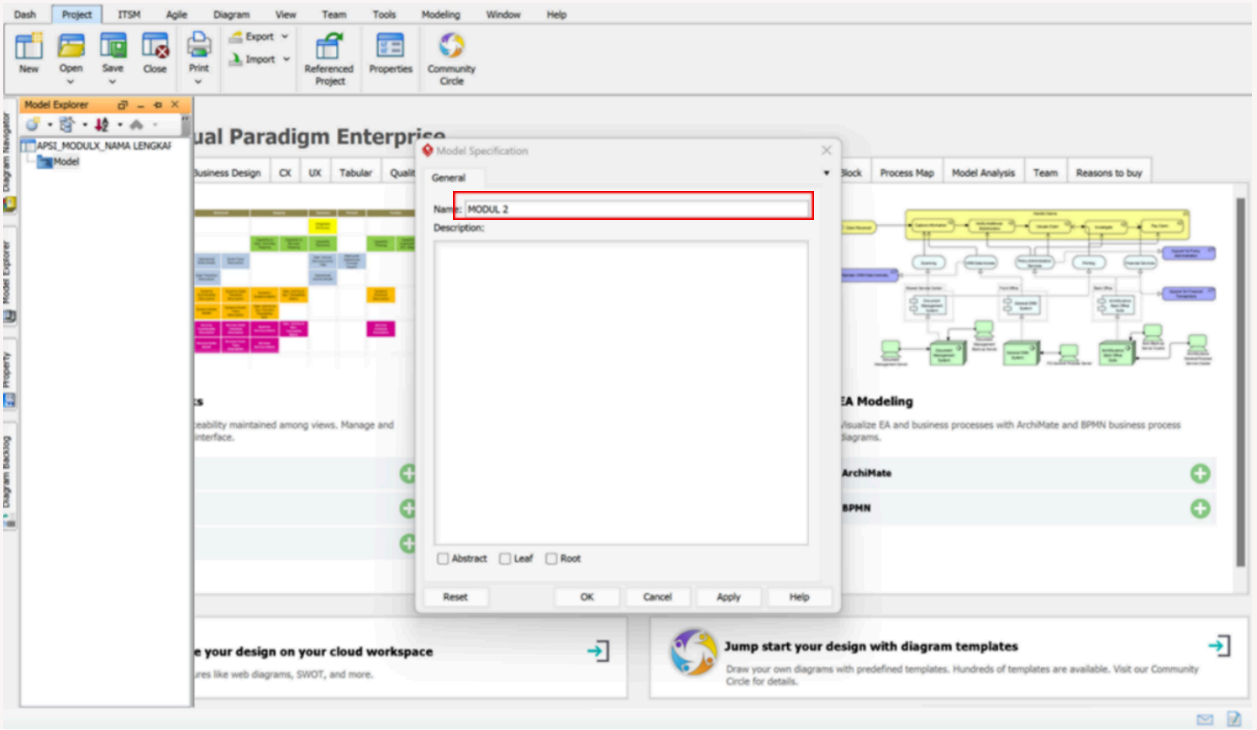
4. Buka Model Explorer pada sisi kiri halaman.



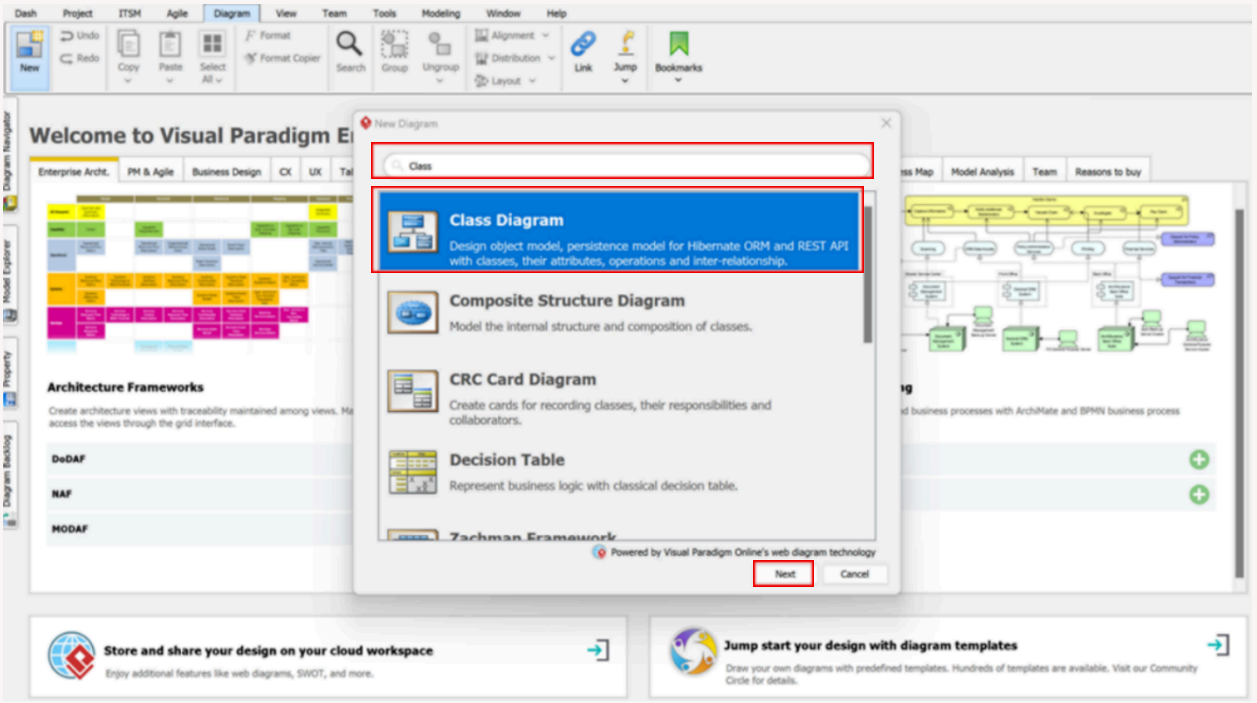
5. Klik kanan pada Project yang telah dibuat > klik Model > klik New Model.



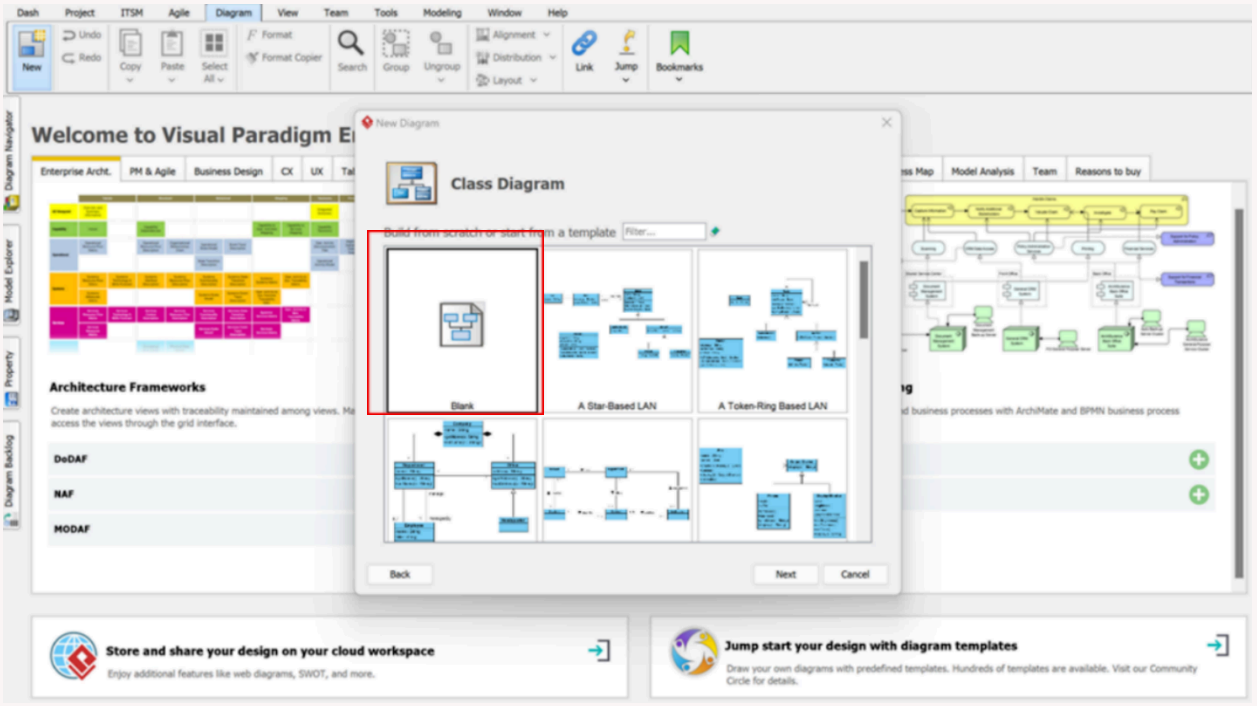
6. Isi Name sesuai modul yang akan dikerjakan.



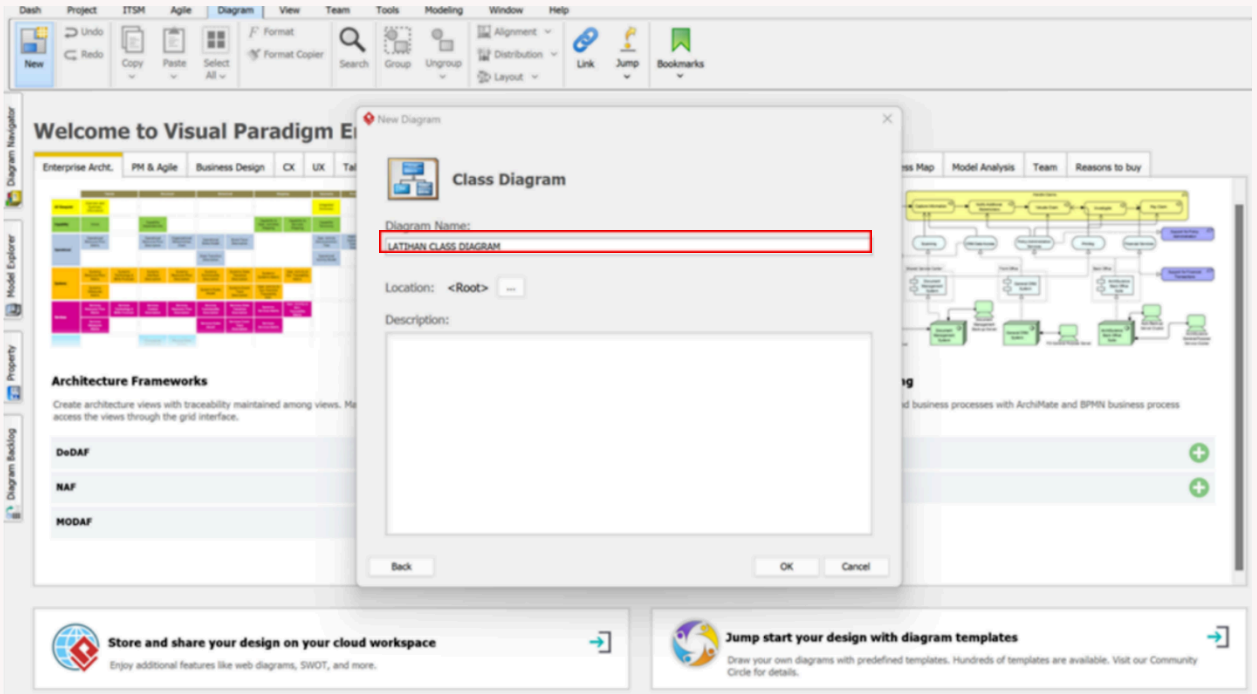
7. Pilih Tab Diagram > pilih New > Cari Class Diagram > Klik Next.



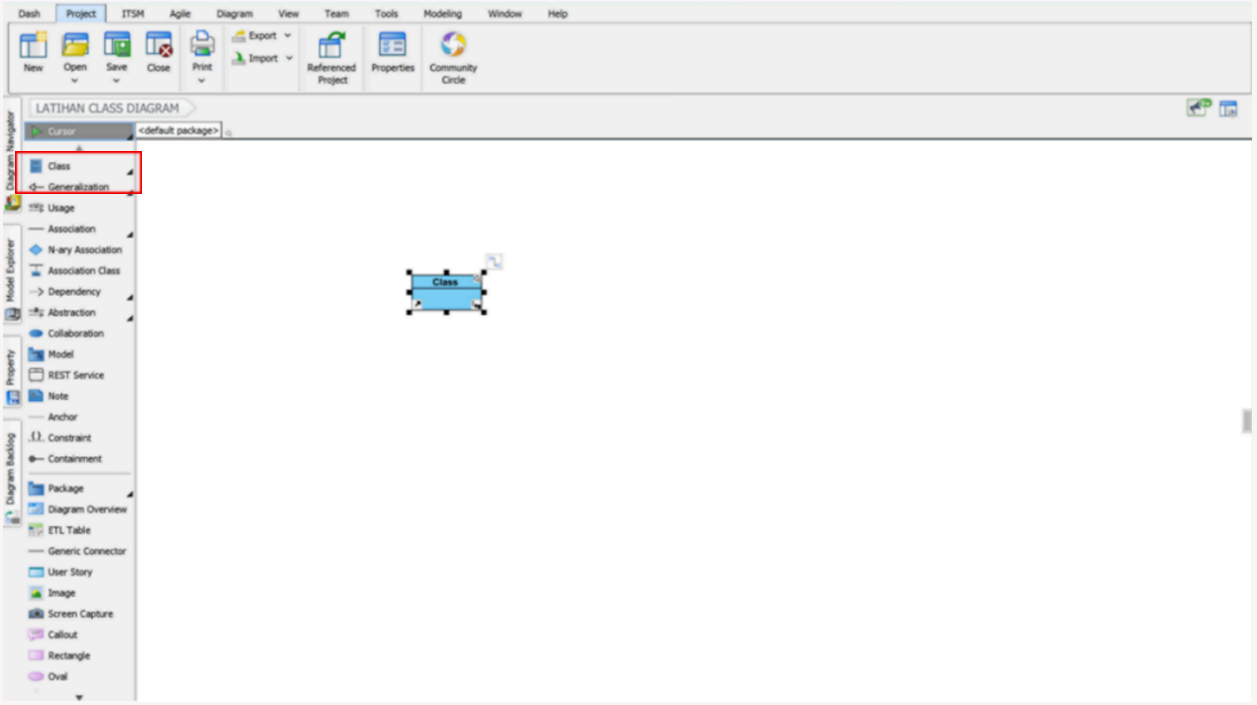
8. Pilih Blank.



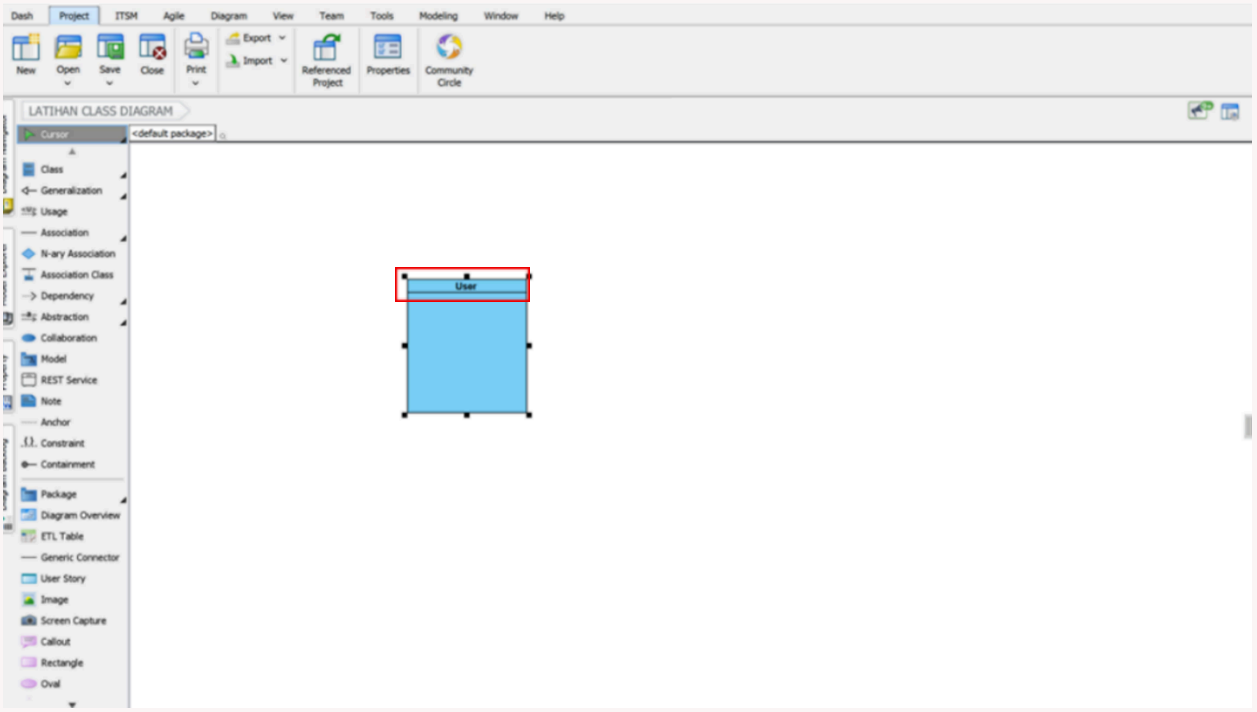
9. Isi Name sesuai ketentuan.



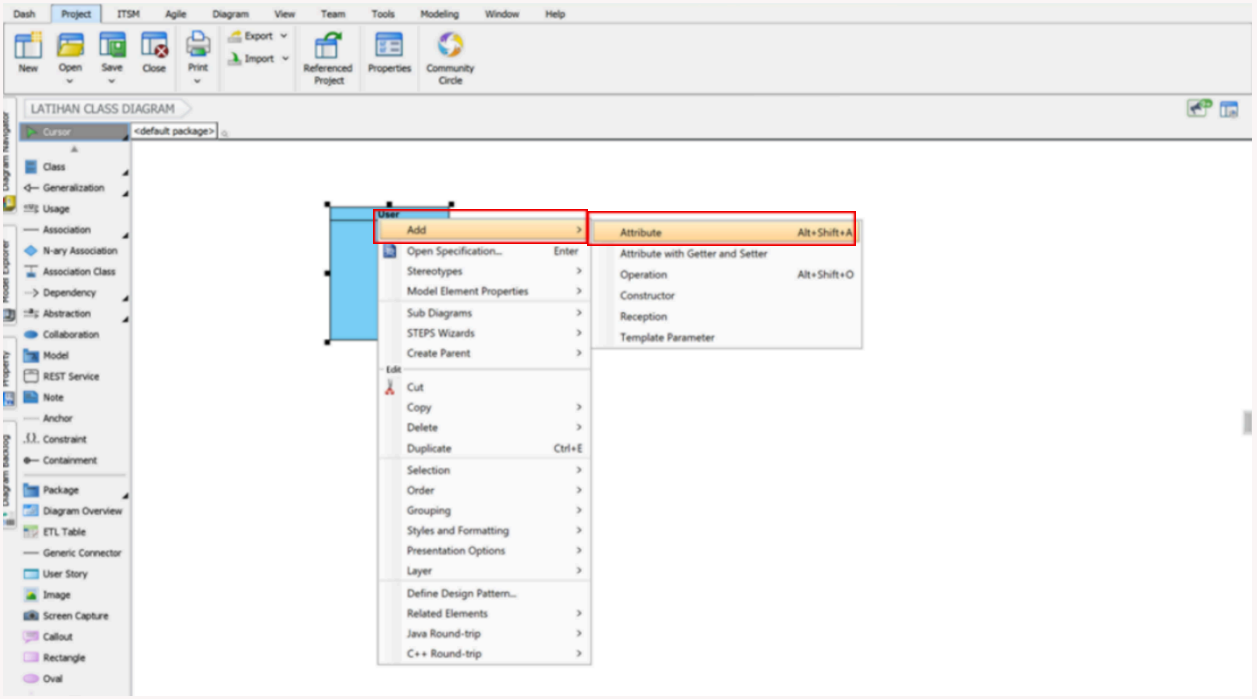
10. Klik Class.



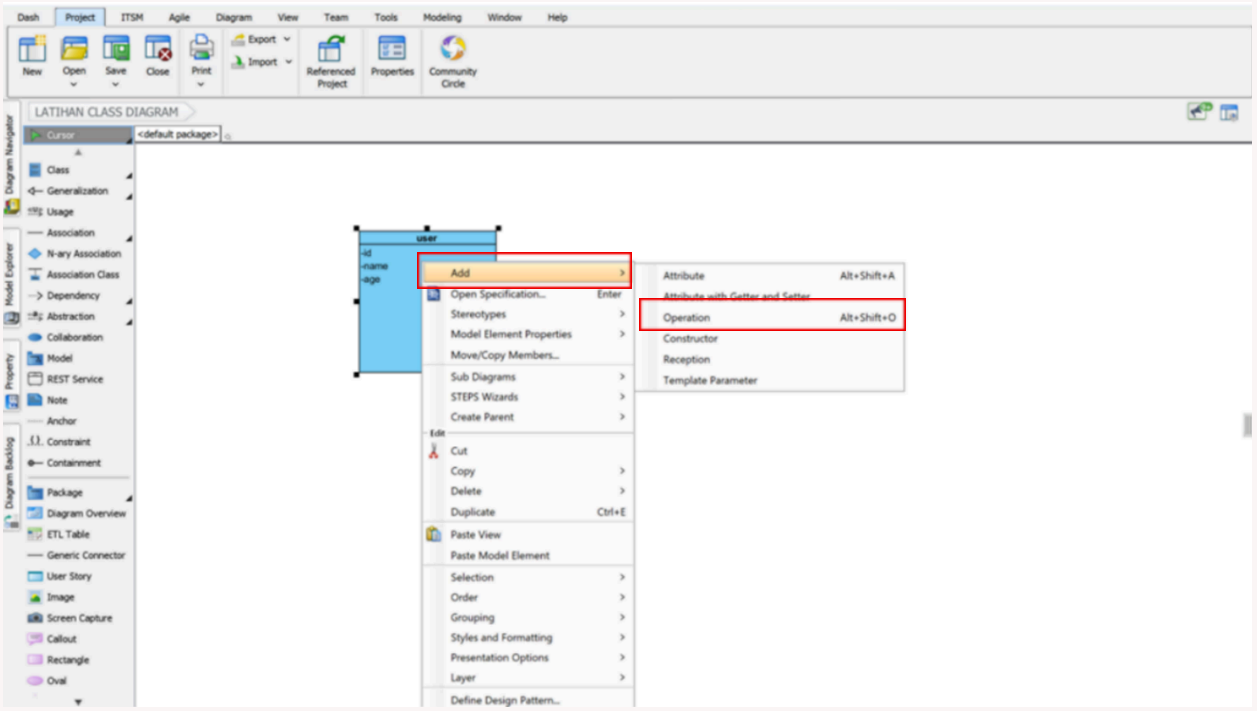
11. Klik dua kali pada nama Class dan ubah nama Class sesuai dengan ketentuan.



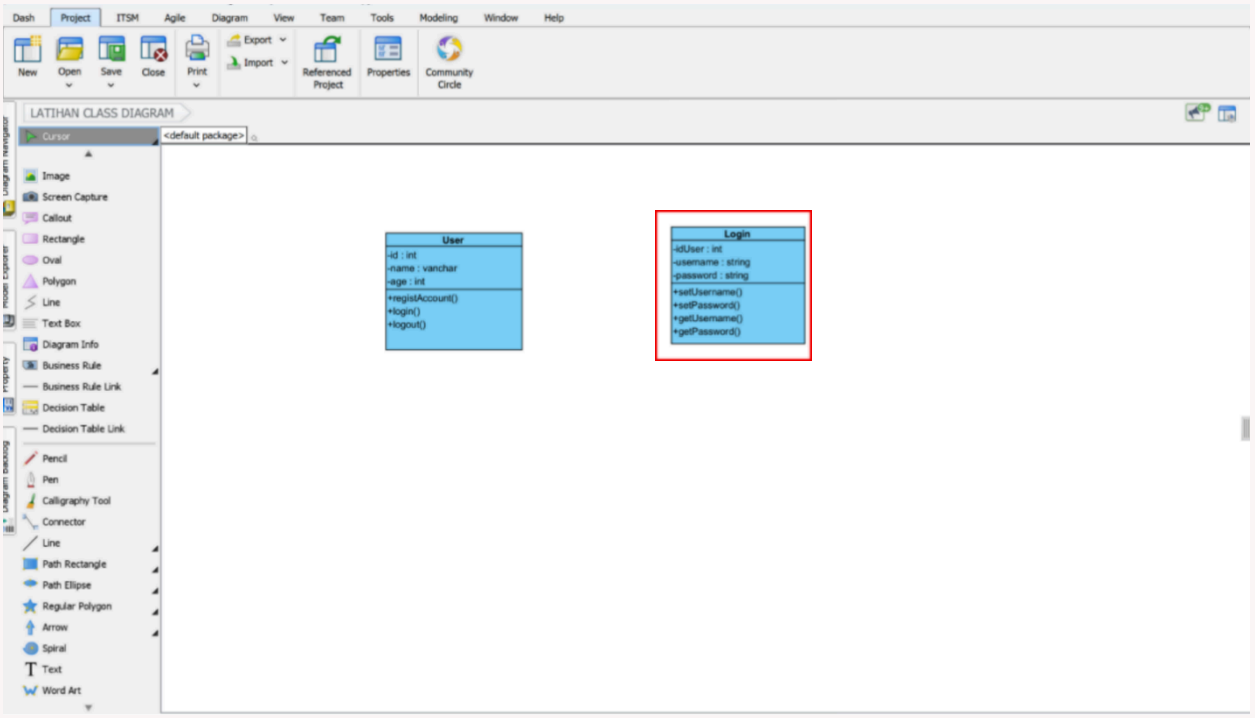
12. Klik kanan pada Class > Klik Add > Klik Atribute untuk menambahkan atribut.



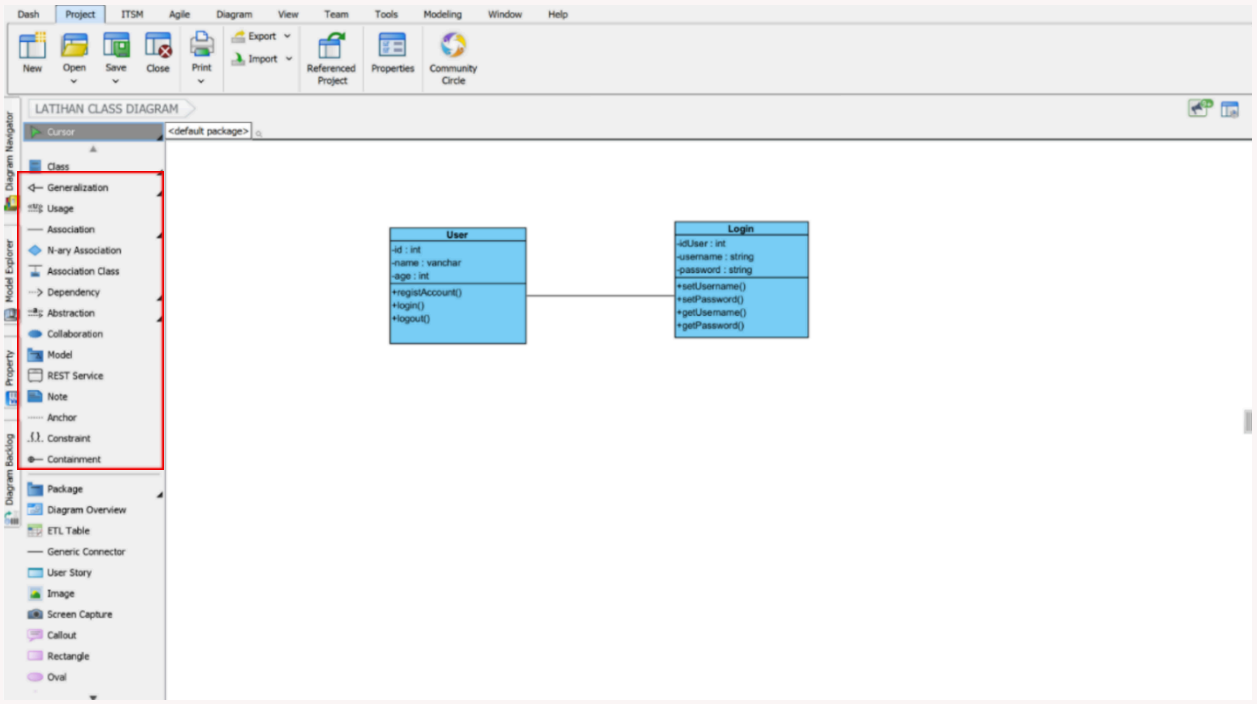
13. Klik kanan pada Class > Klik Add > Klik Operation untuk menambahkan operasi.



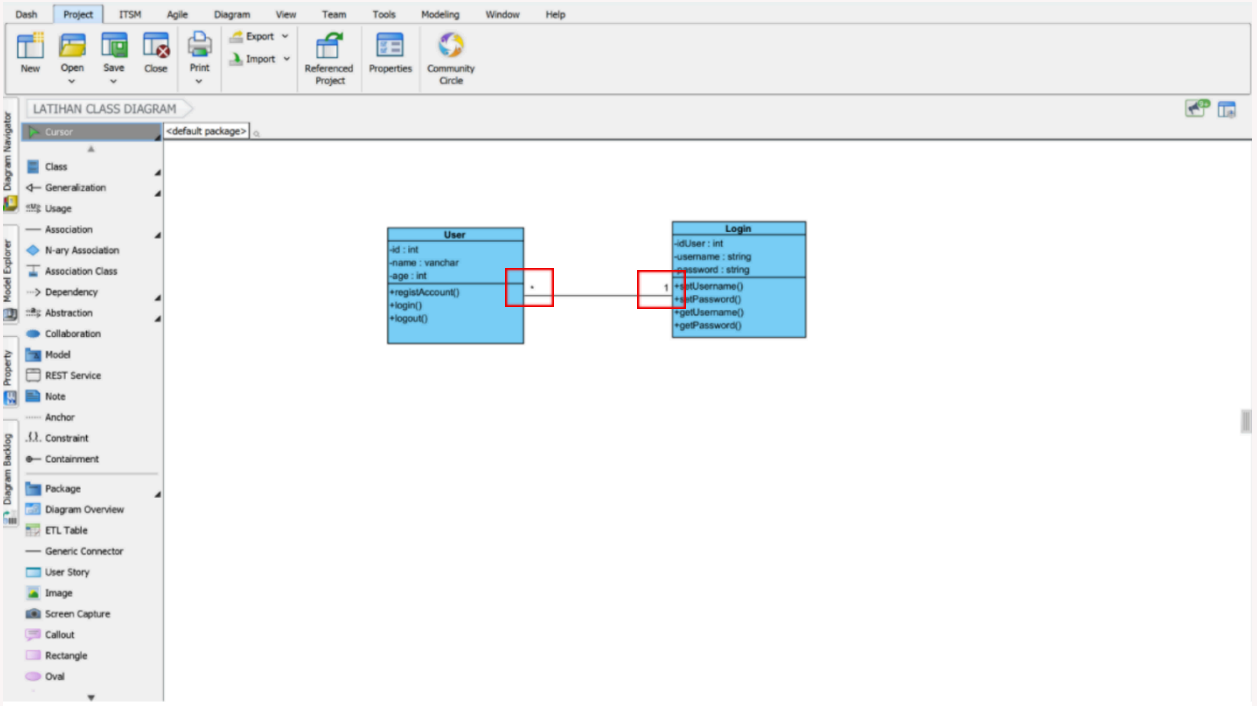
14. Buat Class diagram yang lain sebelum lanjut ke poin berikutnya.



15. Pilih relasi yang sesuai untuk menghubungkan antar Class.

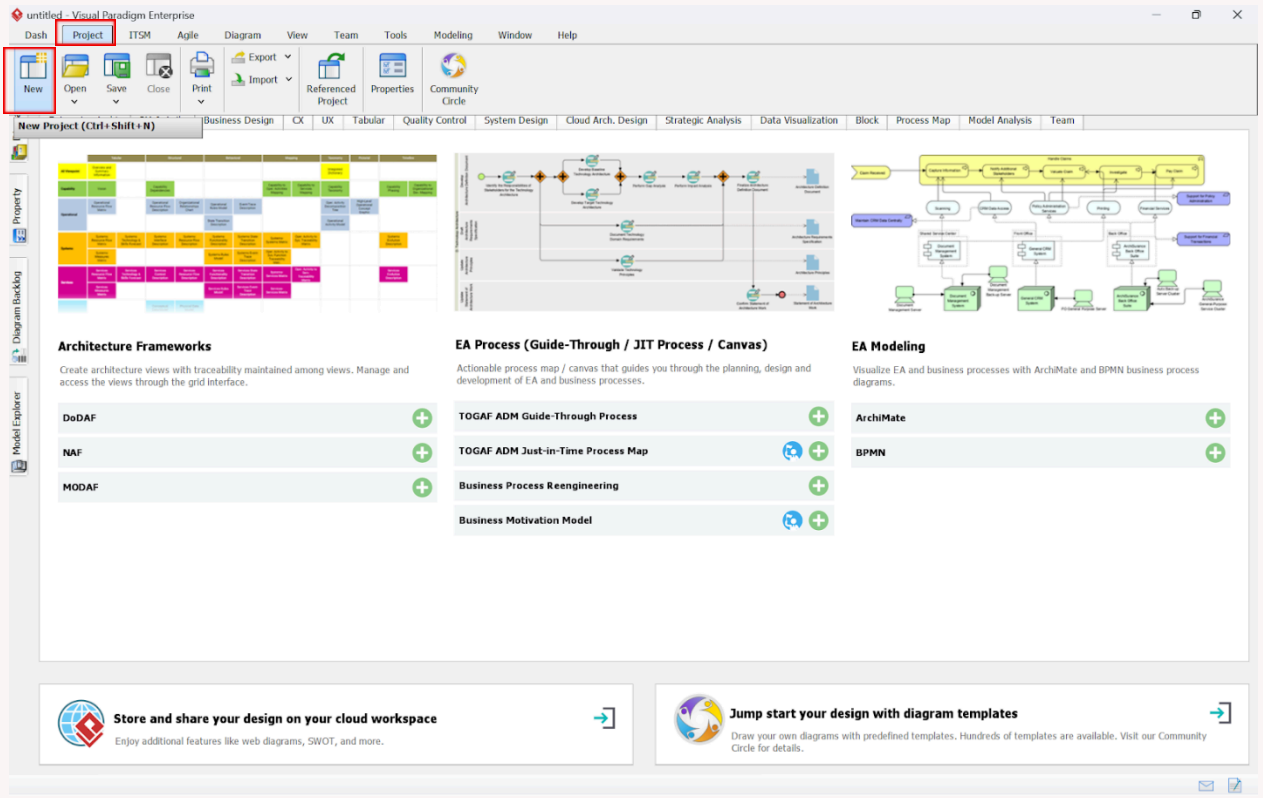


16. Double klik pada setiap ujung relasi untuk memasukan Kardinalitas pada setiap Class yang terhubung.

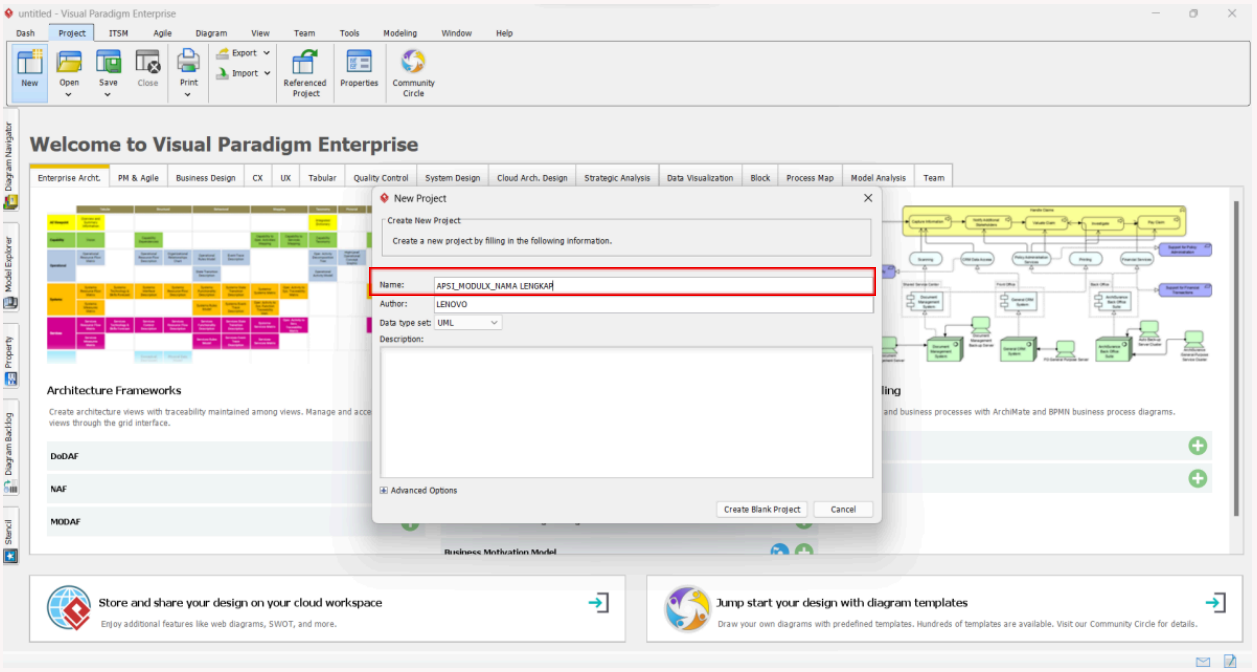


B. Cara Membuat Sequence Diagram

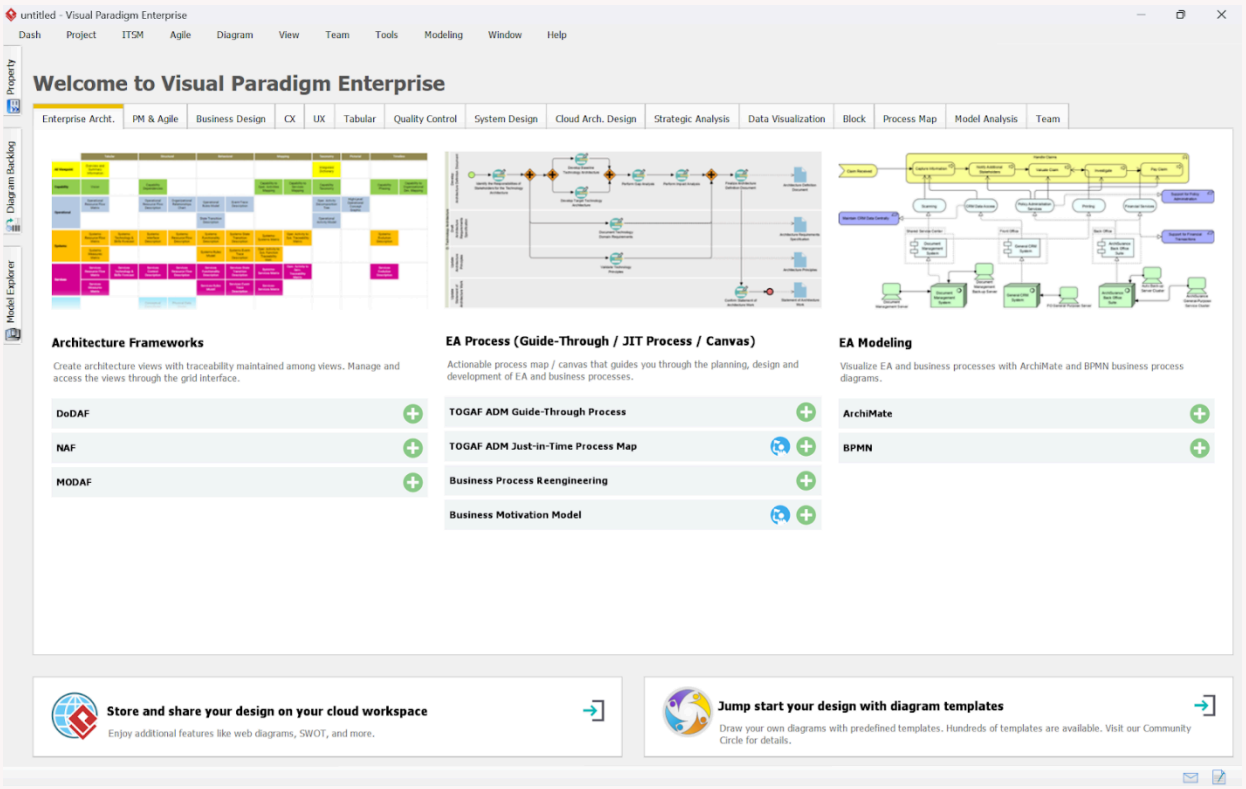
1. Pada tampilan awal *Dashboard* Visual Paradigm, klik *Project* lalu klik *New*.



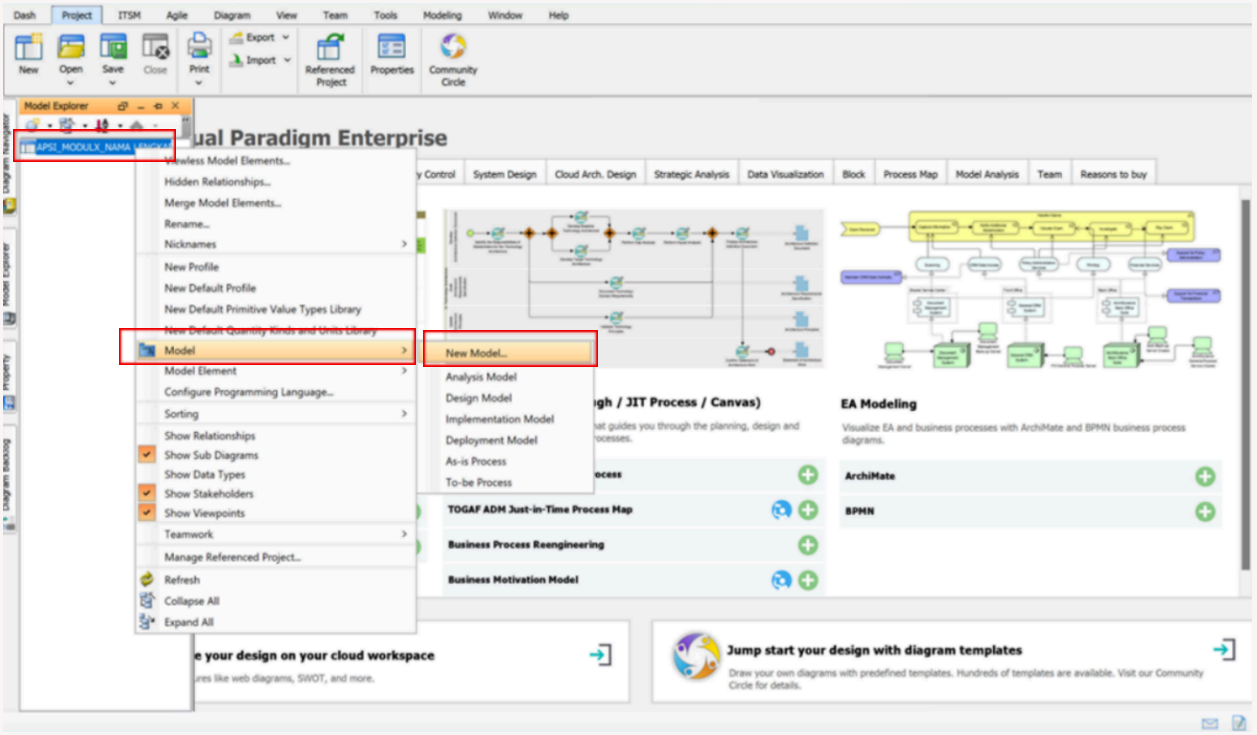
2. Isi nama sesuai ketentuan kemudian klik *Create Blank Project*.



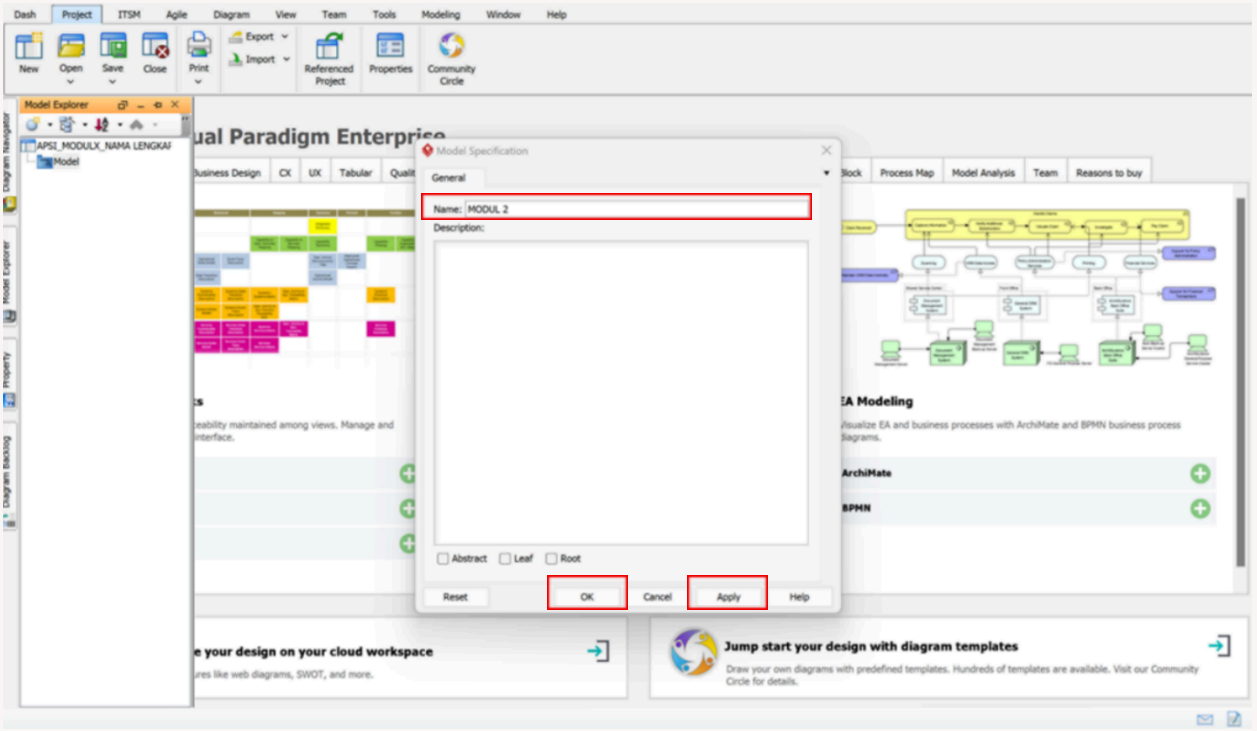
3. Buka Model *Explorer* pada sisi kiri halaman.



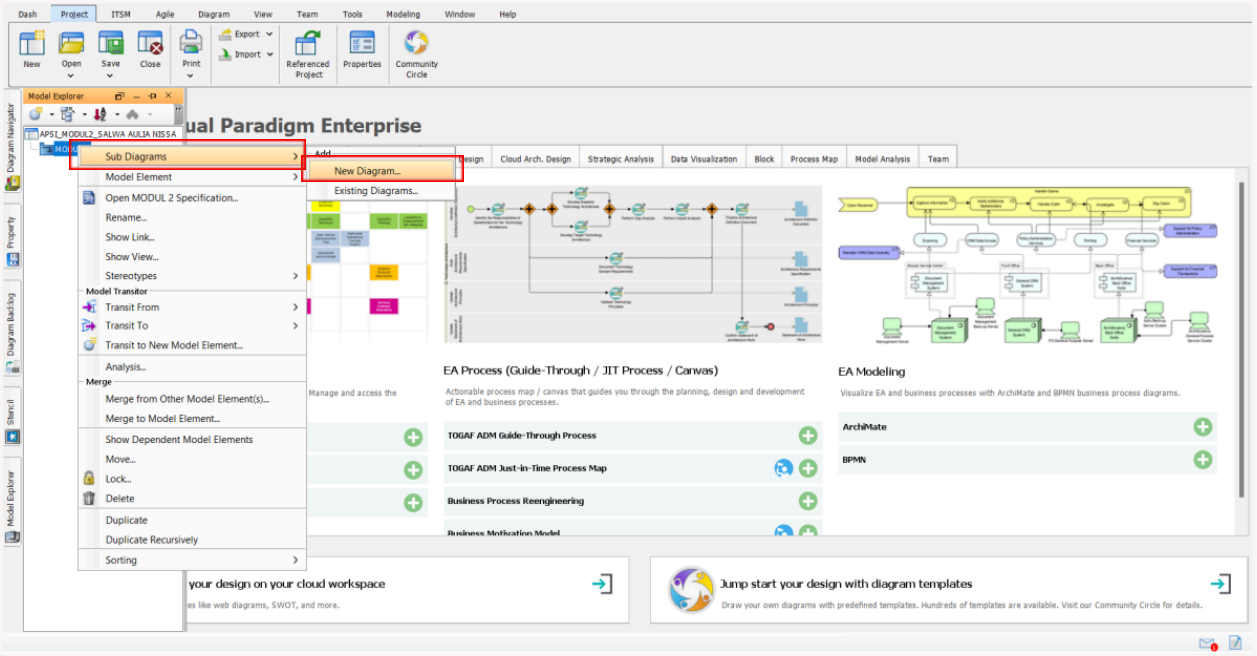
4. Klik kanan pada *Project* yang telah dibuat > klik *Model* > klik *New Model.*



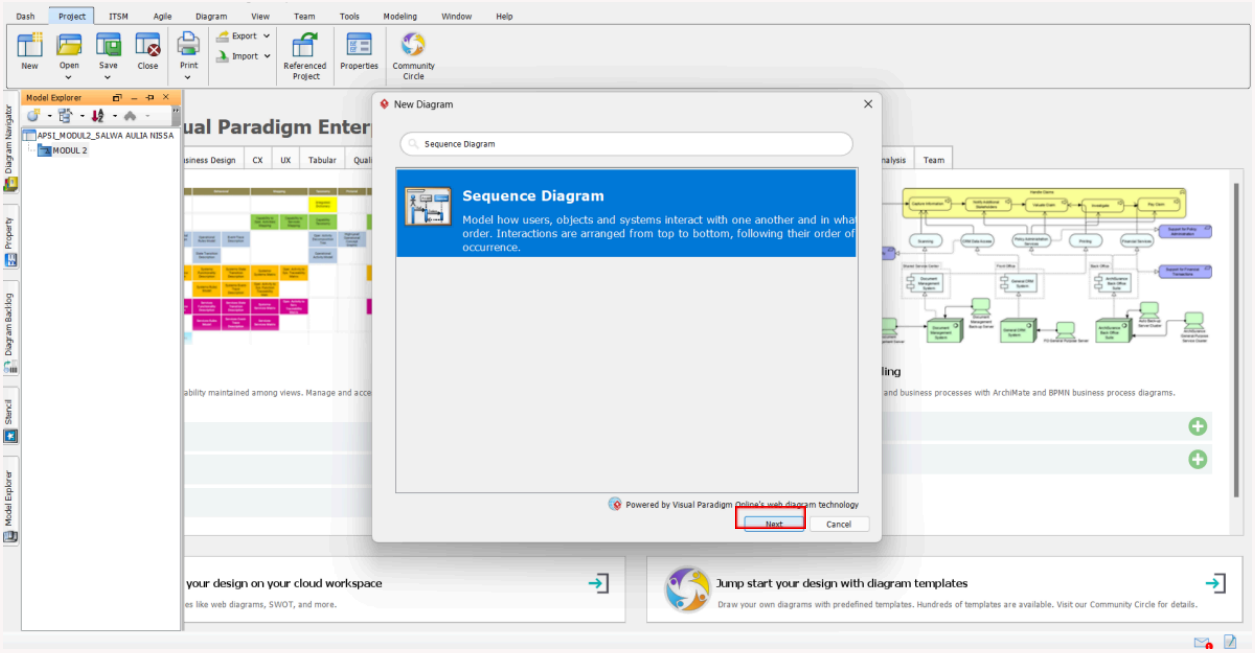
5. Isi *Name* sesuai dengan modul yang dikerjakan, lalu klik *Apply,* lalu klik *OK.*



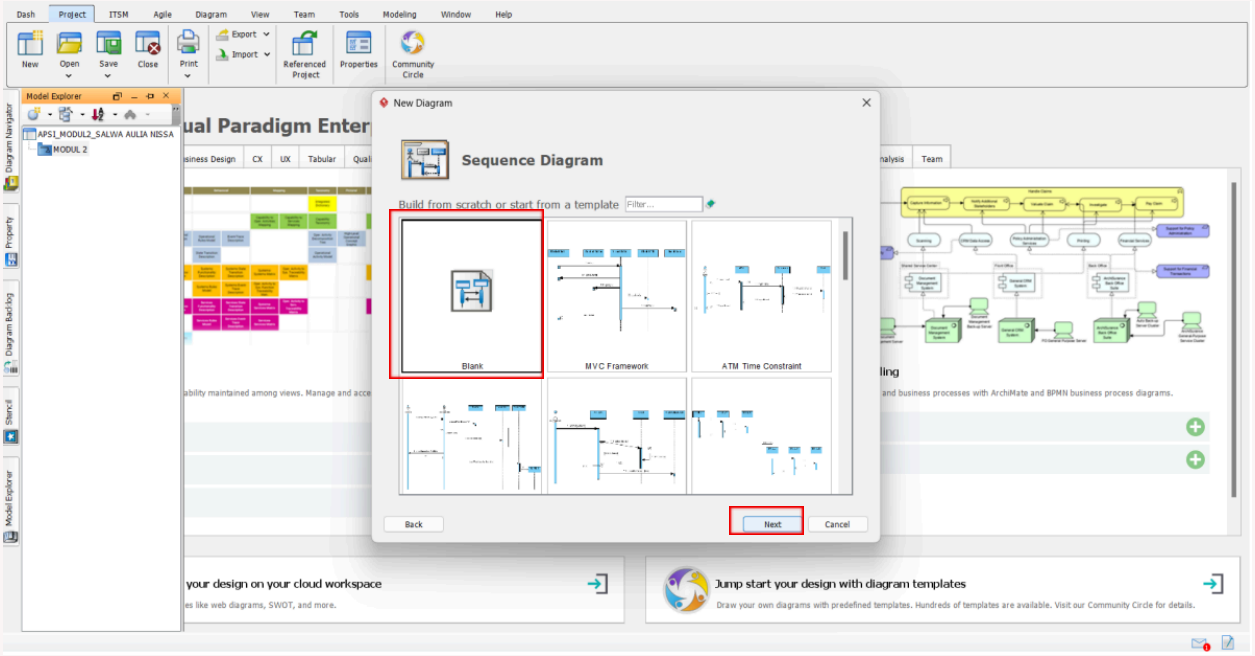
6. Klik kanan pada model yang telah dibuat > klik *Sub* Diagram > *New Diagram.*



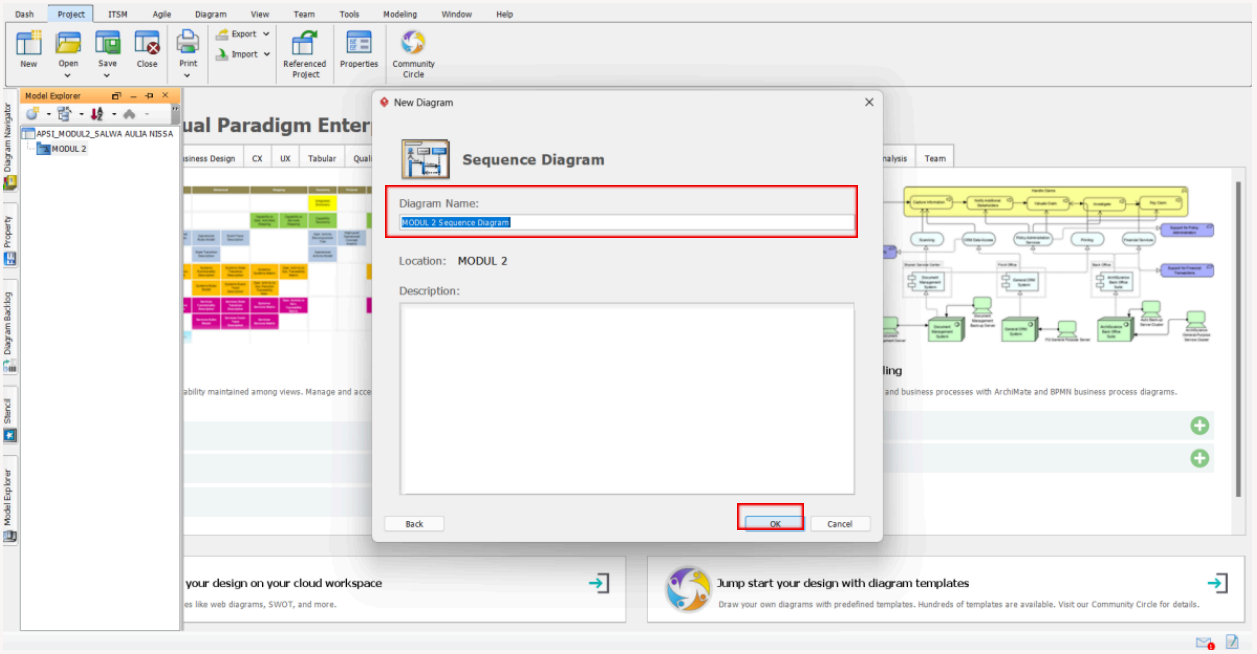
7. Cari *"Sequence Diagram"* di pencarian, lalu klik *Next*.



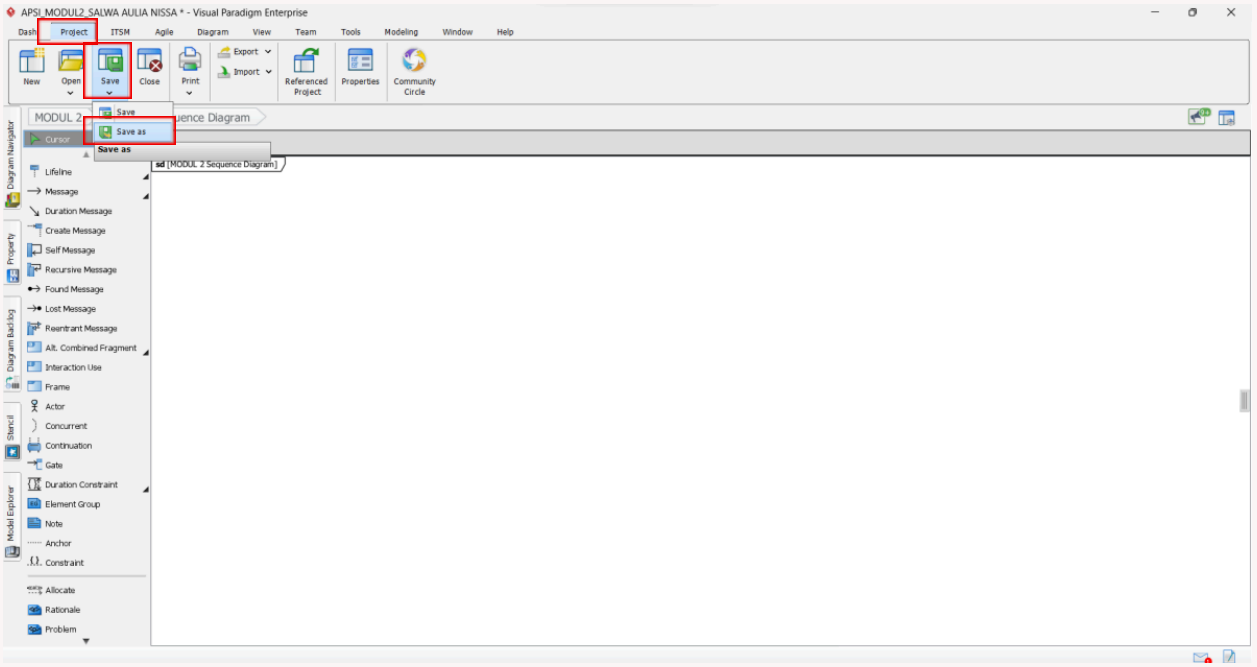
8. Klik *"Blank"* lalu *klik Next*



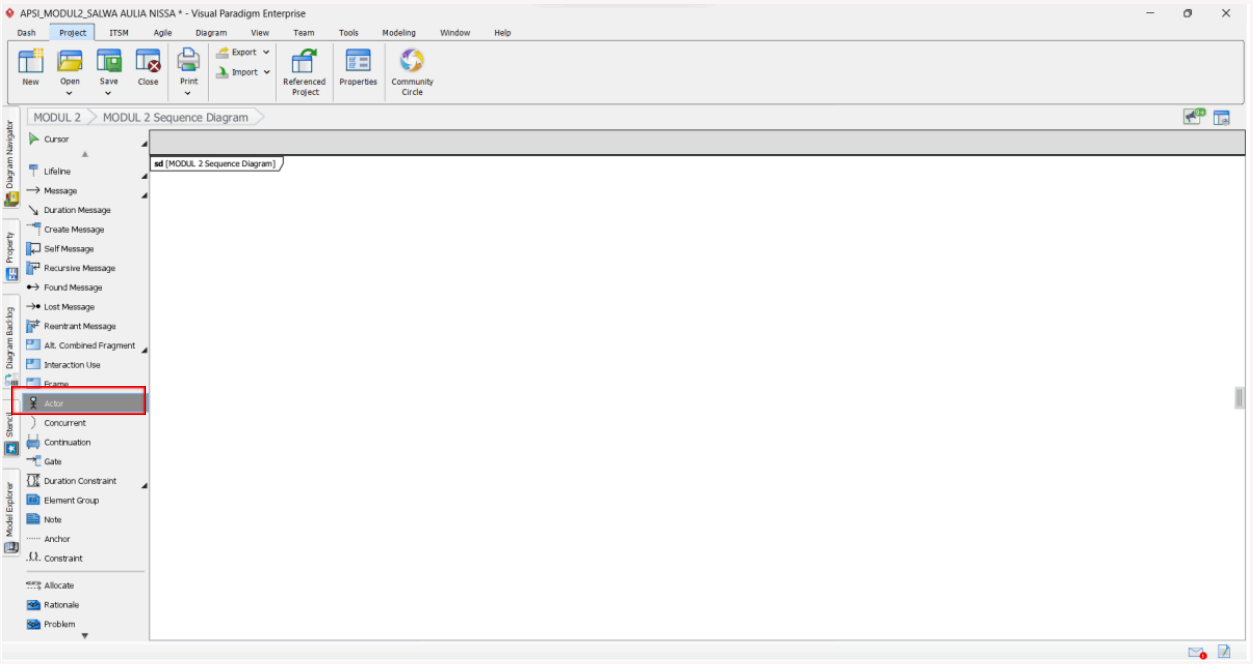
9. Berikan nama sesuai modul dan *diagram* yang dikerjakan. Lalu klik *Ok.*

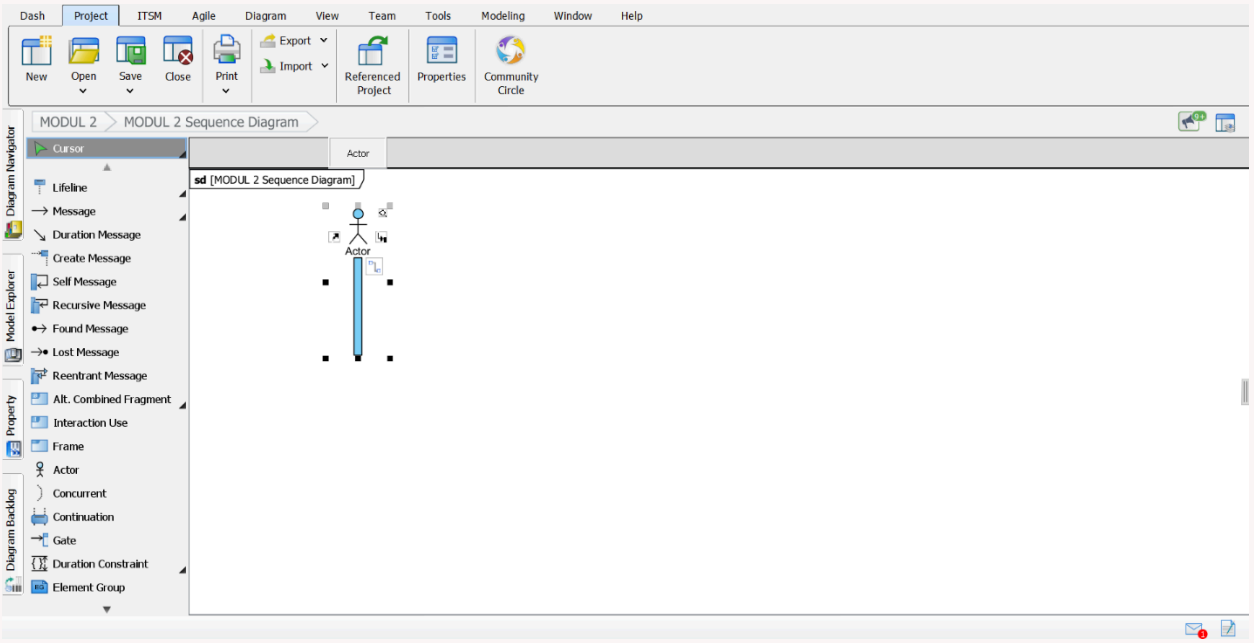


10. Sebelum memulai pengerjaan, pastikan menyimpan pada laptop masing - masing, dengan klik *Project* > *Save as.*

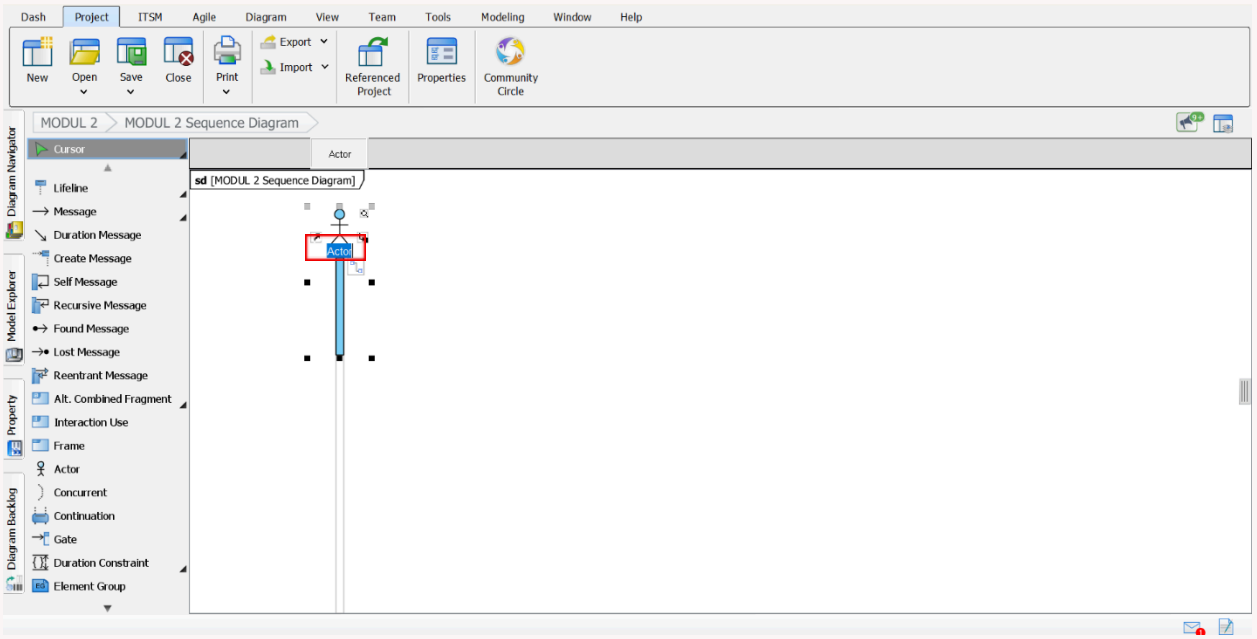


11. Dari *toolbar* pilih *Actor* dan lakukan *drag and drop* untuk ditambahkan ke *worksheet*.

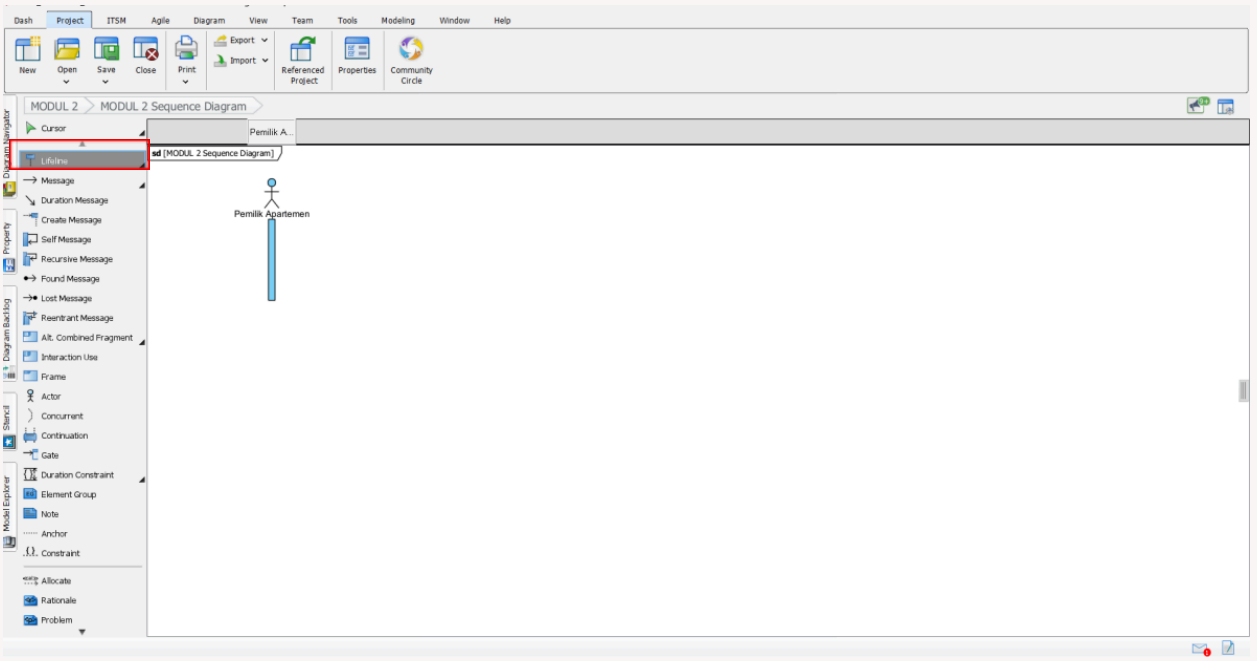


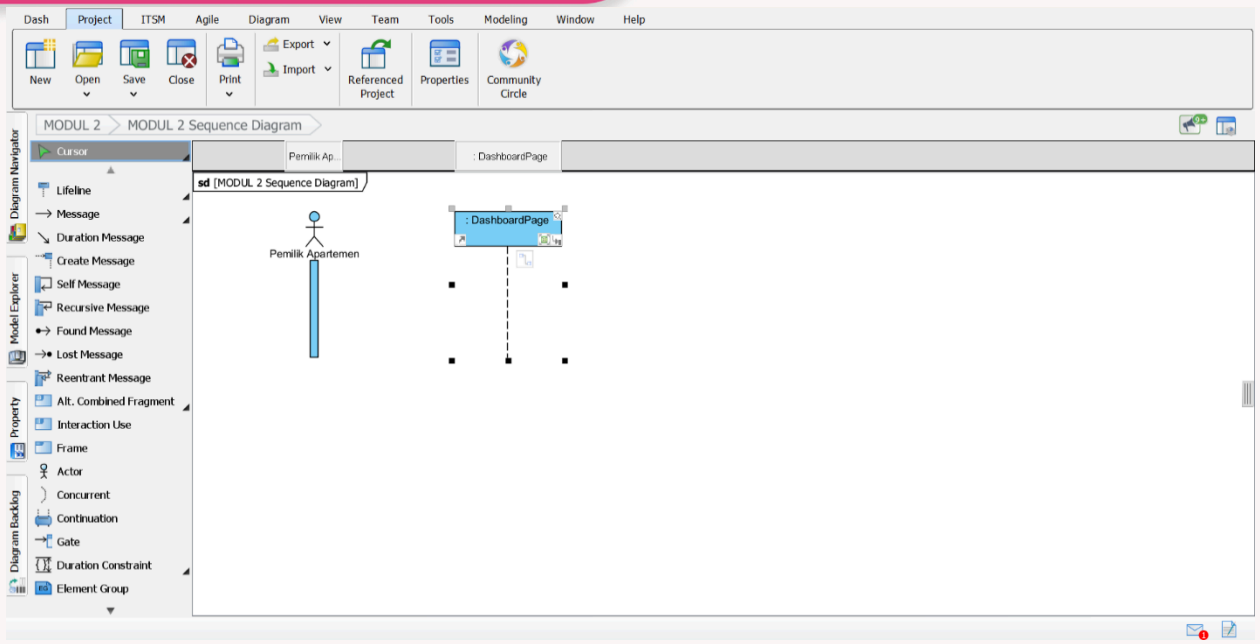


12. Untuk mengganti nama *Actor*, *double-click Actor* dan ganti namanya sesuai skenario studi kasus.

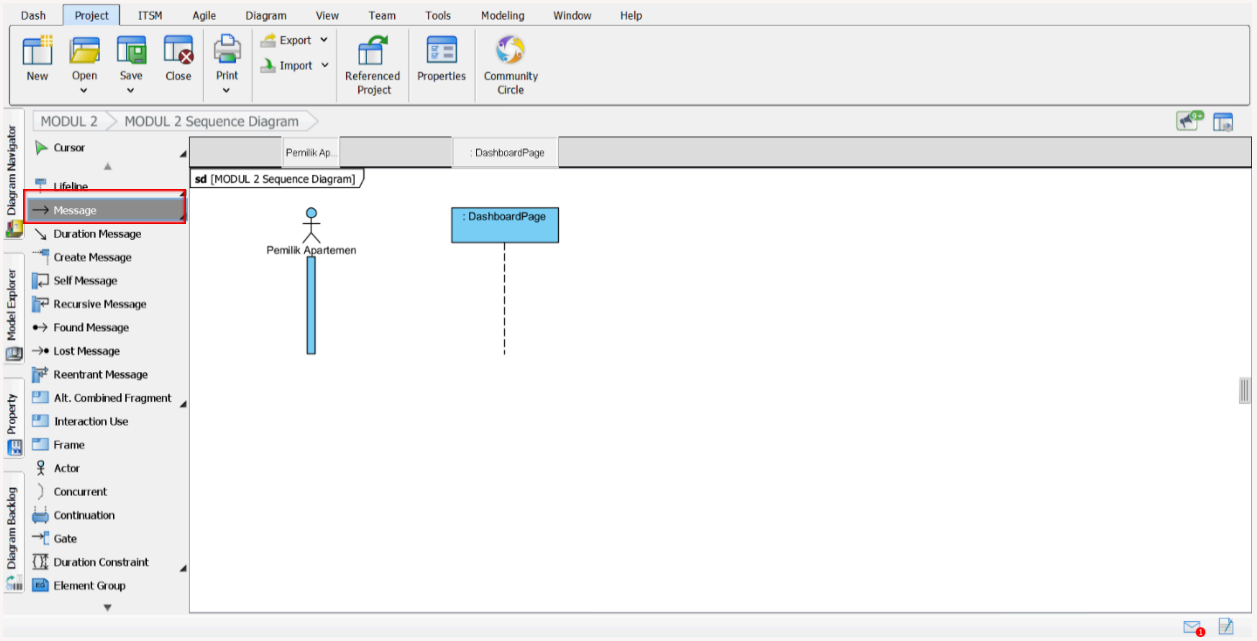


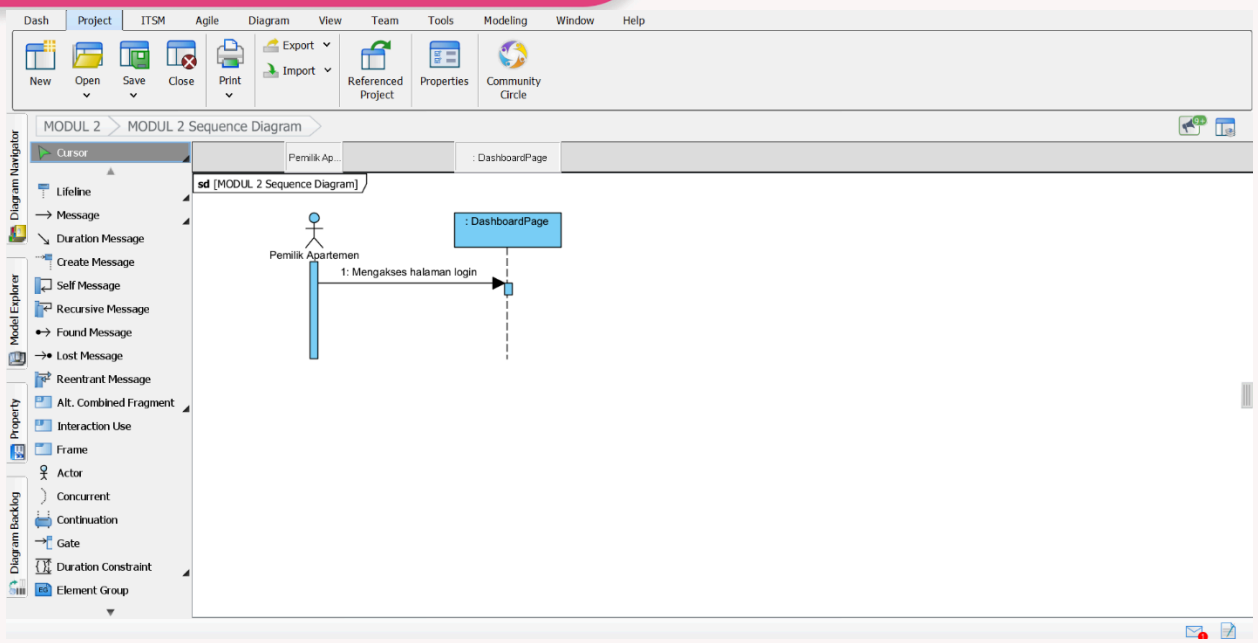
13. Tambahkan *Lifeline* dengan melakukan *drag and drop* dari *toolbar* ke *worksheet*. Untuk mengganti nama *lifeline*, *double-click* nama *lifeline* dan ganti namanya sesuai skenario studi kasus.



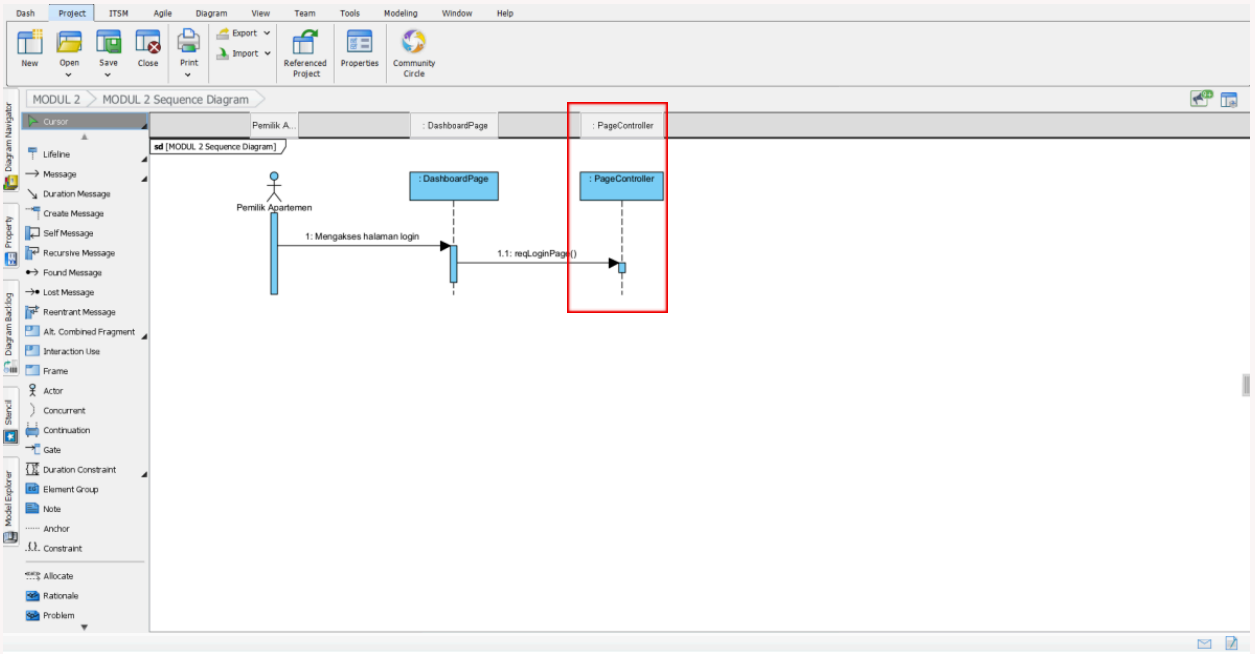


14. Klik *Message* pada *toolbar*, lalu lakukan *drag and drop* dari *Actor* ke *lifeline* yang dituju, lalu tambahkan nama pada *message* sesuai studi kasus.

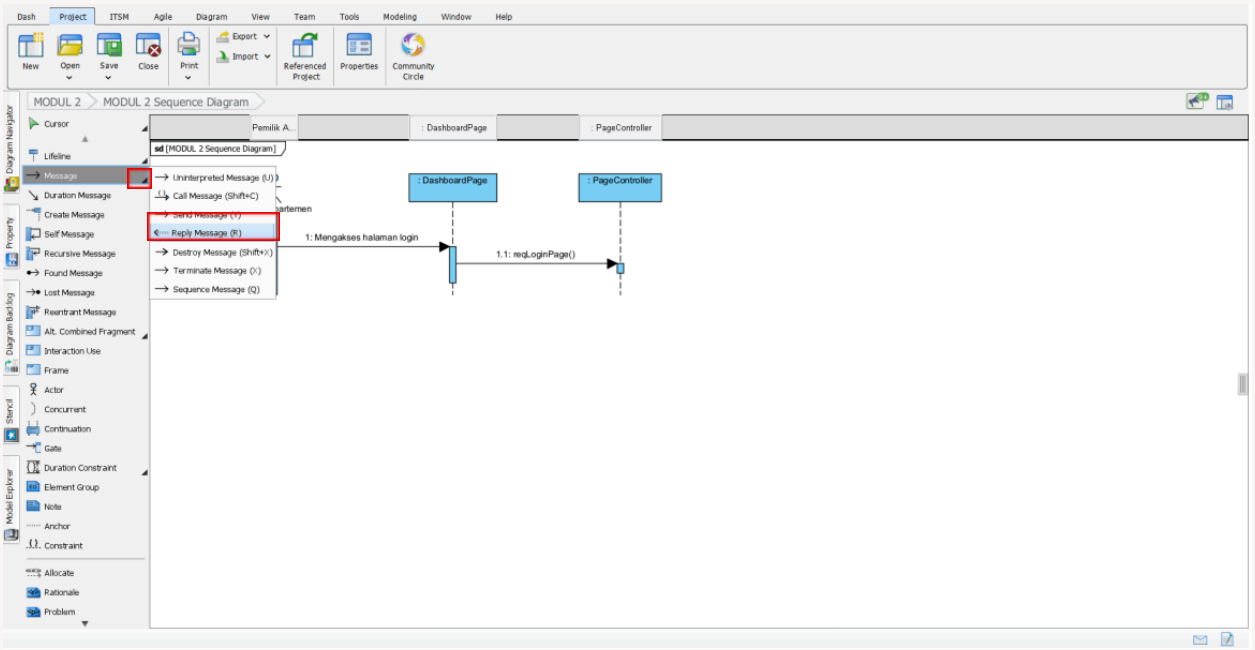




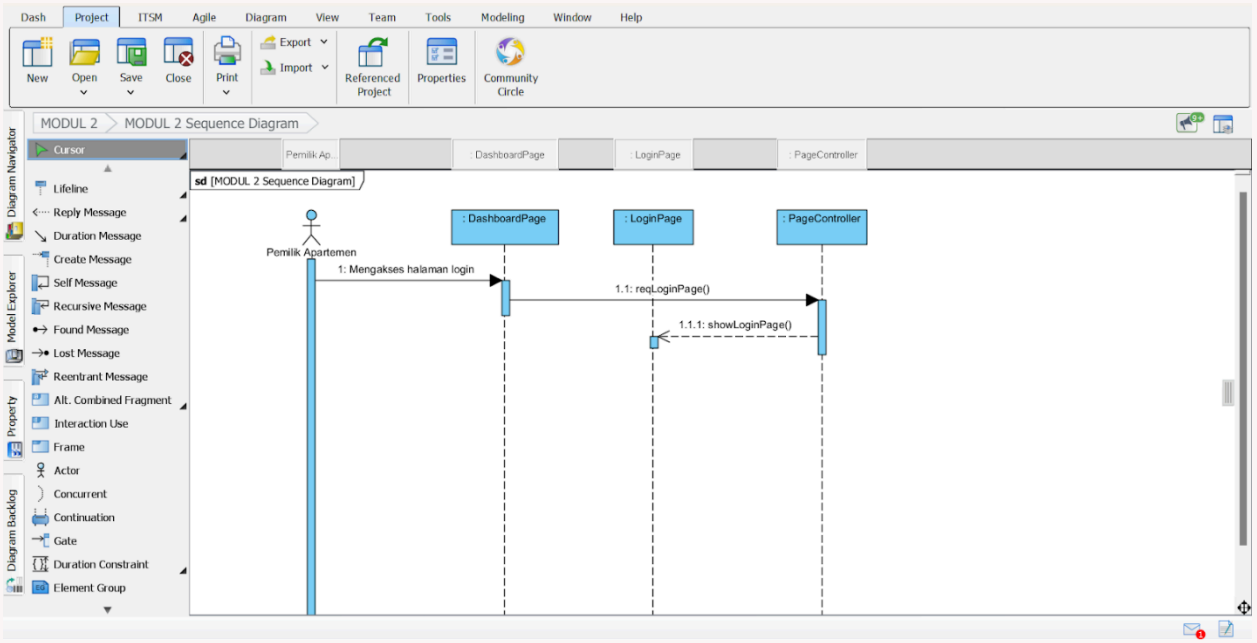
15. Tambahkan *Lifeline Controller* sebagai objek mediasi antara *Boundary* dan *Entity*.



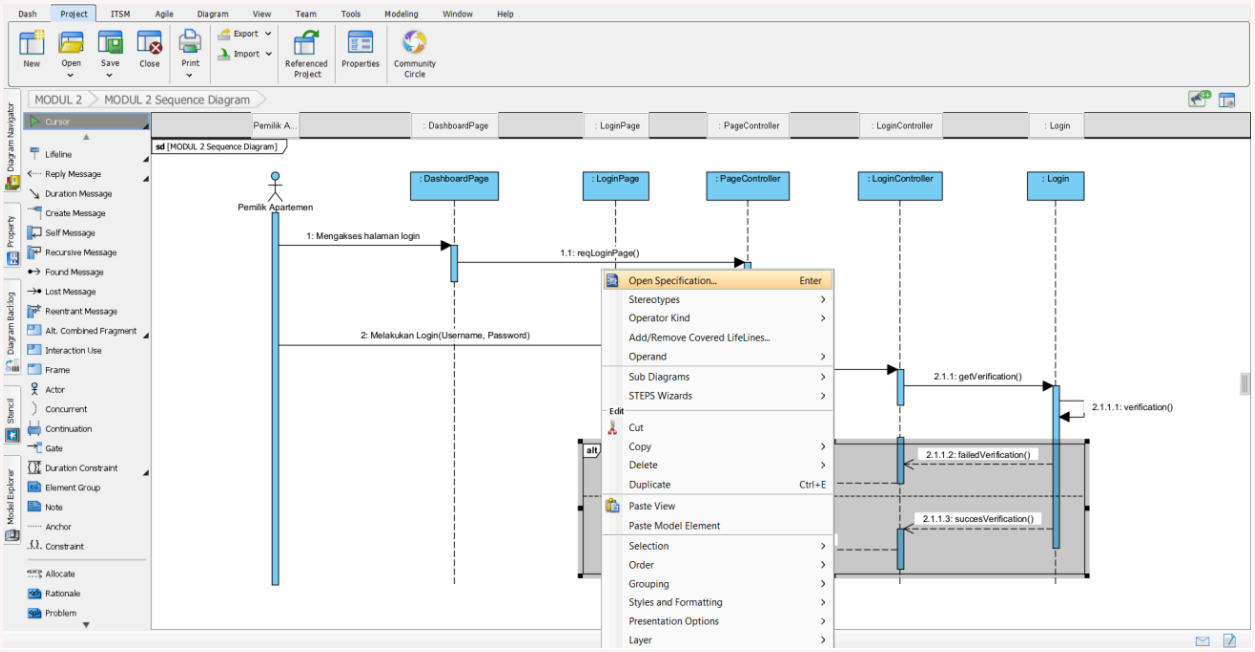
16. Tambahkan *Reply Message* untuk menambahkan timbal balik dari pemanggilan *method* dengan cara klik tanda segitiga siku-siku yang berada pada ujung *message* yang berada pada *toolbar* dan lakukan *drag and drop* notasi *replay message.*



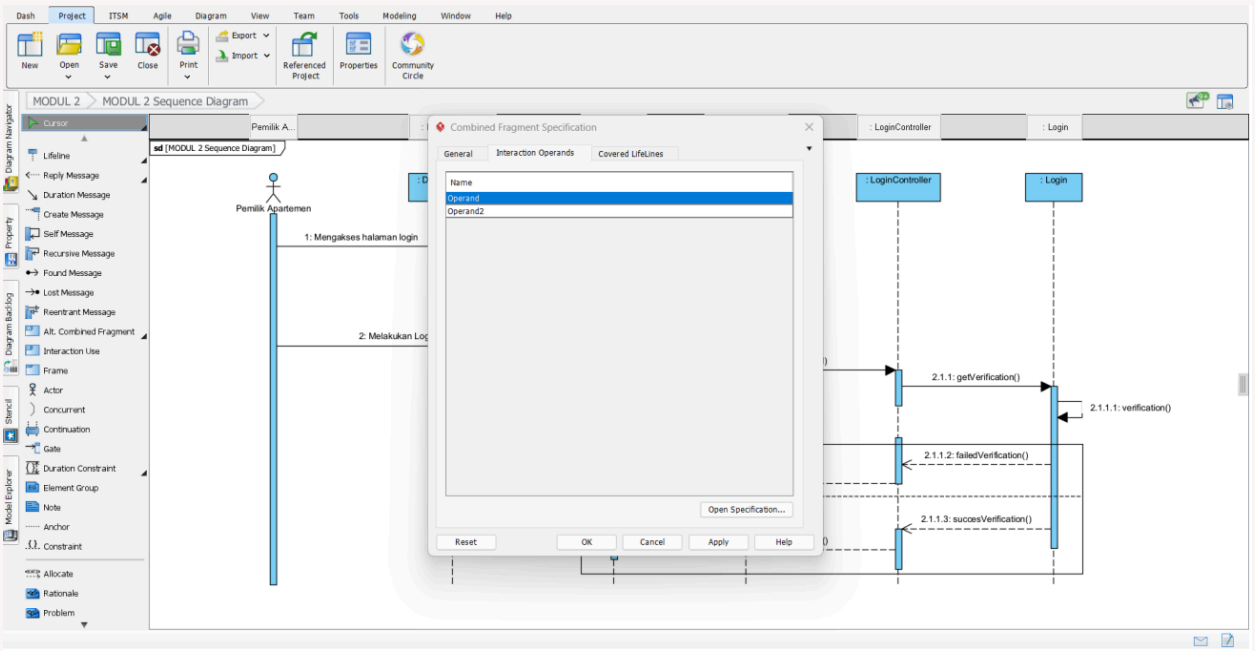
17. Lakukan *drag and drop notasi reply message* yang ada pada *toolbar* untuk menambahkan notasi yang diperlukan pada *sequence diagram.*



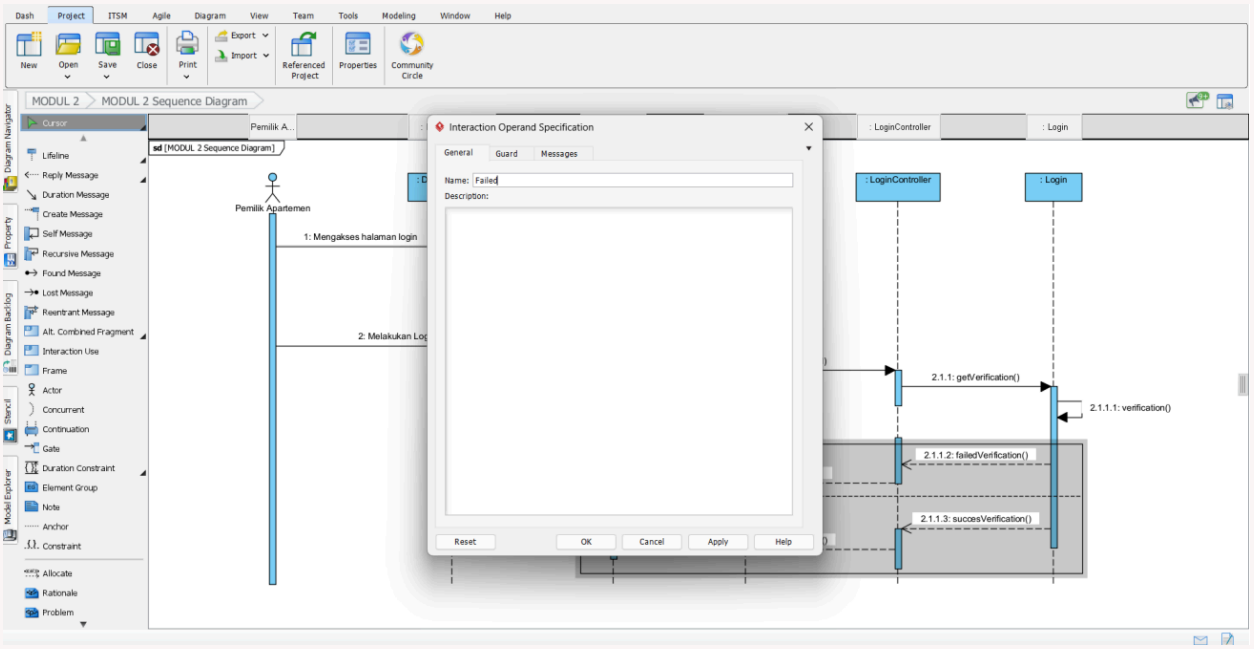
18. Jika ingin menambahkan *frame Alt. Combined Fragment,* lakukan *drag and drop* untuk menambahkan notasi pada *toolbar*. Tambahkan kondisi dengan melakukan klik kanan pada notasi *Alt. Combined Fragment* dan klik *Open Specification*.

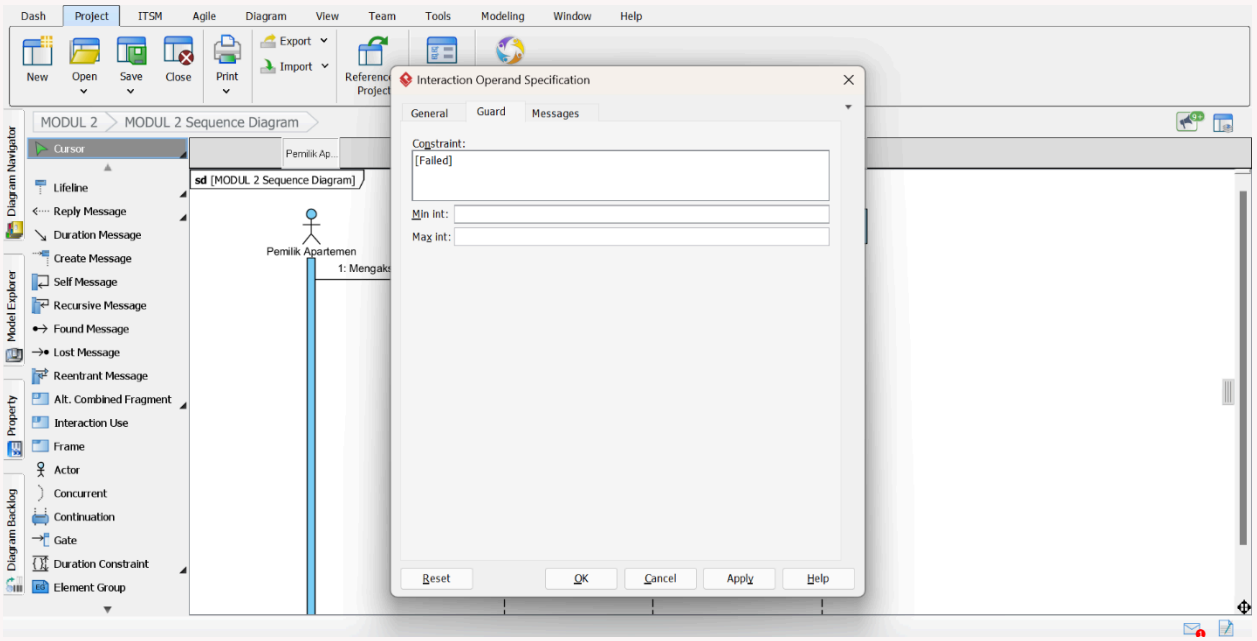


19. Klik dua kali *Operand* pada *tab Interaction Operands.*

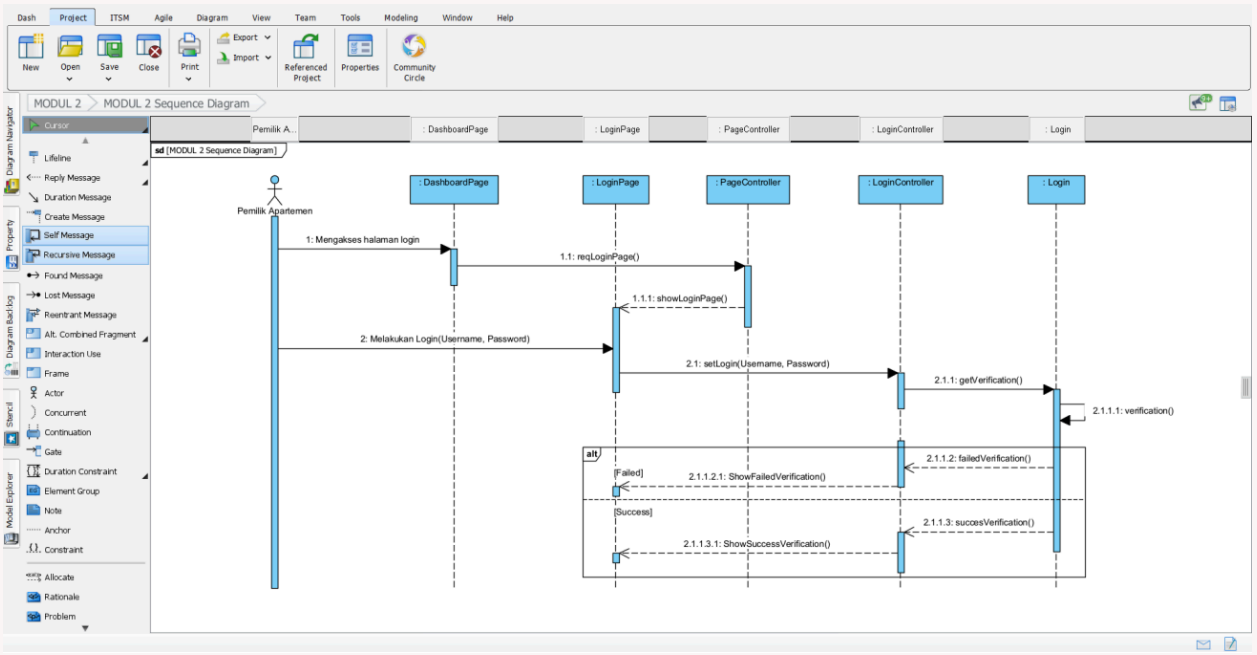


20. Ganti *Name* pada *tab General* dan *Constraint* pada *tab Guard* sesuai dengan kondisi studi kasus. Lalu klik OK. Lakukan langkah yang sama untuk *Operand2*. Setelah itu klik OK untuk menyelesaikan pengaturan.





21. Berikut adalah tampilan hasil dari pengaturan *Open Spesification* yang telah dilakukan pada notasi frame *Alt. Combined Fragment*.



CONTOH

A. Studi Kasus 1 (Level Mudah)

EAD Bookstore adalah sebuah usaha toko buku online yang dimiliki oleh seorang pengusaha independen. Usaha ini menyediakan berbagai jenis buku untuk masyarakat umum, termasuk pelajar, mahasiswa, pekerja profesional, maupun masyarakat umum yang gemar membaca. EAD Bookstore dikenal karena koleksi bukunya yang bervariasi, mulai dari buku pelajaran, literatur, novel, hingga buku-buku referensi profesional, serta layanan pemesanan yang praktis tanpa harus datang ke toko fisik.

Seiring dengan meningkatnya penggunaan internet dalam kehidupan sehari-hari, EAD Bookstore berencana untuk mengembangkan sistem manajemen penjualan buku yang lebih modern. Sistem ini diharapkan dapat memanfaatkan teknologi informasi untuk mempermudah seluruh proses bisnis, mulai dari pencarian, pemesanan, pembayaran, hingga pelacakan status pengiriman buku. Dengan adanya sistem ini, EAD Bookstore dapat menjangkau pelanggan di berbagai lokasi tanpa batasan jarak maupun waktu.

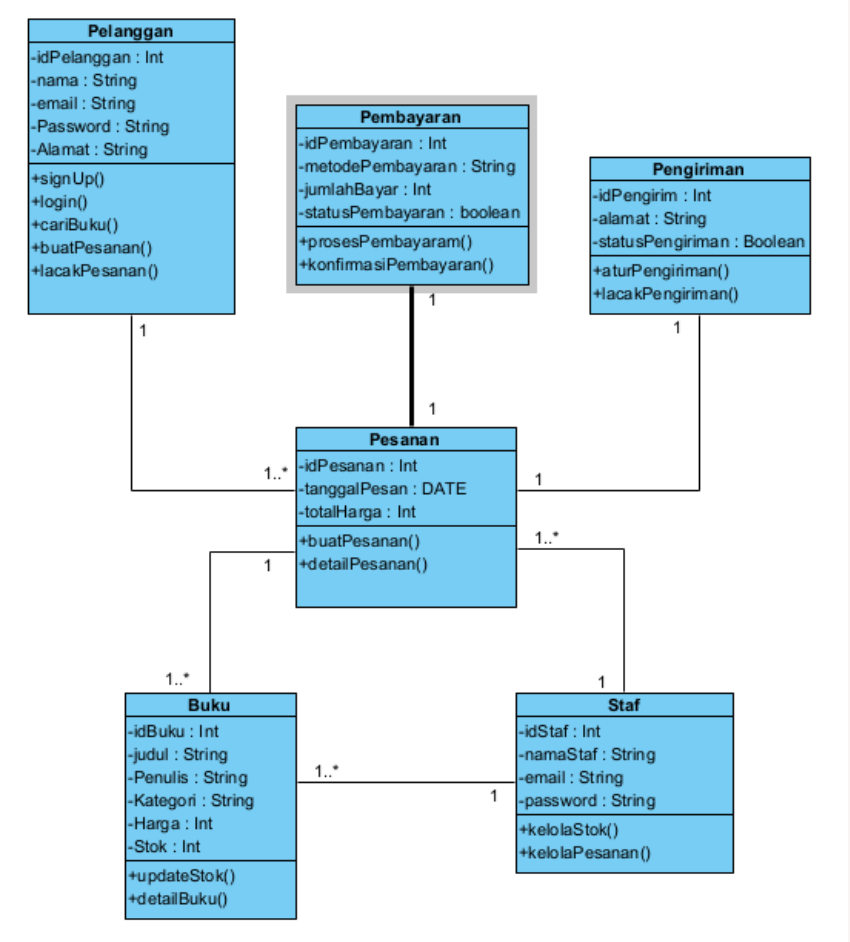
Sistem yang akan dibangun adalah aplikasi berbasis web yang dapat diakses oleh pelanggan dan staf toko. Dari sisi pelanggan, sistem EAD Bookstore dirancang untuk memberikan kemudahan dalam membeli buku dari mana saja tanpa harus datang langsung ke toko. Salah satu fitur utama adalah pendaftaran akun pelanggan, di mana pelanggan baru dapat dengan mudah membuat akun untuk mengakses berbagai layanan yang ditawarkan. Setelah mendaftar, pelanggan dapat melakukan pencarian buku berdasarkan judul, penulis, atau kategori tertentu. Hasil pencarian dapat langsung dimasukkan ke dalam keranjang belanja, sehingga pelanggan bisa memilih beberapa buku sekaligus sebelum melanjutkan ke proses pemesanan.

Pada tahap berikutnya, pelanggan dapat melakukan pembayaran online dengan berbagai metode yang tersedia, seperti transfer bank, kartu kredit, atau dompet digital. Setelah pembayaran berhasil diverifikasi, sistem akan secara otomatis membuat data pesanan dan memberi nomor transaksi. Pelanggan juga dapat dengan mudah melacak status pengiriman buku mereka melalui web, mulai dari tahap konfirmasi pesanan, proses pengemasan, hingga pengiriman oleh jasa kurir. Fitur ini memberikan transparansi yang tinggi bagi pelanggan sehingga pelanggan selalu mengetahui kapan buku mereka akan tiba di alamat tujuan.

Dari sisi staf toko, sistem ini juga memberikan manfaat besar. Manajemen stok buku dapat dilakukan secara real-time, sehingga jumlah ketersediaan buku di katalog online selalu ter-update sesuai dengan kondisi gudang. Staf toko dapat menambahkan buku baru, memperbarui informasi buku, atau menghapus buku yang sudah tidak tersedia. Selain itu, staf juga dapat memantau daftar pesanan pelanggan, memproses pengemasan, serta memastikan bahwa setiap pesanan dikirim tepat waktu.

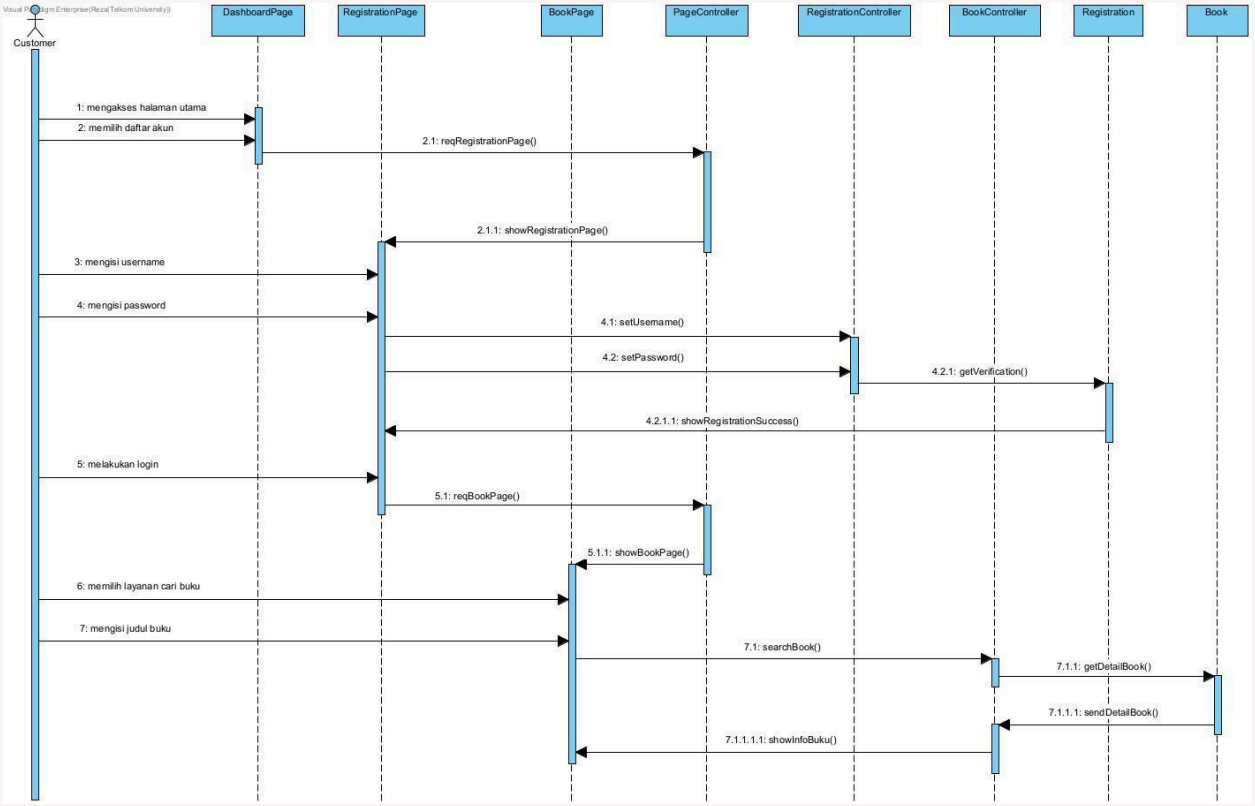
Dengan semua fitur ini, EAD Bookstore berkomitmen untuk memberikan layanan yang cepat, efisien, dan dapat diakses dari mana saja. Sistem ini bukan hanya memudahkan pelanggan dalam berbelanja buku secara online, tetapi juga meningkatkan efisiensi operasional dari sisi pengelolaan toko. EAD Bookstore percaya bahwa dengan dukungan teknologi informasi, mereka dapat bersaing dengan toko buku lain dan memberikan pengalaman berbelanja yang lebih modern, nyaman, dan terpercaya bagi semua pelanggan.

1. Class Diagram



2. Sequence Diagram

- Berikut contoh Sequence Diagram pada fitur Pencarian Buku



B. Studi Kasus 2 (Level Kompleks)

EAD University, sebagai salah satu perguruan tinggi terbaik, memutuskan untuk memperbarui sistem manajemen pembelajarannya dengan membangun platform digital yang terintegrasi sepenuhnya. Tujuannya adalah untuk meningkatkan efisiensi proses pembelajaran bagi mahasiswa, dosen, dan admin yang terlibat dalam sistem akademik.

Dalam beberapa tahun terakhir, EAD University telah mengalami lonjakan jumlah mahasiswa yang signifikan. Dengan semakin berkembangnya teknologi digital, banyak dosen yang mulai mengajarkan mata kuliah mereka secara online, namun sistem yang ada tidak lagi mampu mengakomodasi kebutuhan tersebut. Oleh karena itu, EAD University berencana untuk membangun sebuah platform yang memudahkan dosen untuk mengajar, mahasiswa untuk belajar, dan admin untuk mengelola proses akademik dengan lebih baik.

Dalam platform ini, ada tiga aktor utama yang terlibat, yaitu dosen, mahasiswa, dan admin. Dosen tugasnya adalah sebagai pengajar, memberikan materi, mengatur jadwal kuliah, serta menilai tugas dan ujian. Sedangkan mahasiswa adalah pihak yang menerima materi pembelajaran, mengerjakan tugas, mengikuti ujian, dan mengakses informasi terkait perkuliahan. Admin berperan sebagai pengelola sistem yang memastikan bahwa semua data akademik, termasuk jadwal kuliah, materi pembelajaran, dan penilaian, tercatat dengan baik dan dapat diakses oleh pihak-pihak yang berwenang.

Untuk merancang sistem ini, tim pengembang perlu membuat class diagram yang menggambarkan relasi antar class yang ada dalam sistem. Beberapa class yang perlu ada dalam sistem EAD University ini diantaranya Dosen, Mahasiswa, MataKuliah, JadwalKuliah, MateriPembelajaran, Tugas, Ujian, Nilai, dan Admin. Setiap class ini memiliki relasi satu sama lainnya.

Class JadwalKuliah memiliki ketergantungan yang sangat erat dengan class Dosen. Tanpa adanya dosen yang mengajar, jadwal kuliah tidak akan ada. Dosen berperan penting dalam menentukan kapan dan di mana kuliah diadakan. Ketika seorang dosen ditugaskan untuk mengajar mata kuliah tertentu, sistem akan secara otomatis menghubungkan dosen dengan JadwalKuliah untuk memastikan bahwa semua jadwal kuliah diatur sesuai dengan waktu yang tersedia. Tanpa dosen yang mengajarkan, tidak ada kuliah yang dapat dijadwalkan.

Selanjutnya, class MateriPembelajaran, Tugas, dan Ujian memiliki ketergantungan kuat dengan class MataKuliah. Tanpa mata kuliah, materi pembelajaran, tugas, dan ujian tidak akan ada. Setiap mata kuliah memiliki materi

khusus yang perlu dipelajari oleh mahasiswa, serta tugas dan ujian untuk menilai kemajuan mereka. Tanpa mata kuliah yang terdaftar, tidak akan ada materi yang dapat diakses, tidak ada tugas yang dapat dikerjakan,atau ujian yang dapat dilaksanakan oleh mahasiswa.

Kemudian, class Nilai memiliki ketergantungan yang kuat dengan class Mahasiswa. Nilai hanya ada jika ada mahasiswa yang mengambil mata kuliah tertentu. Setiap mahasiswa yang mengikuti suatu mata kuliah akan diberi nilai berdasarkan hasil tugas dan ujian yang mereka kerjakan. Tanpa mahasiswa, tidak ada yang bisa dinilai, dan sistem tidak akan memiliki data untuk mengeluarkan nilai.

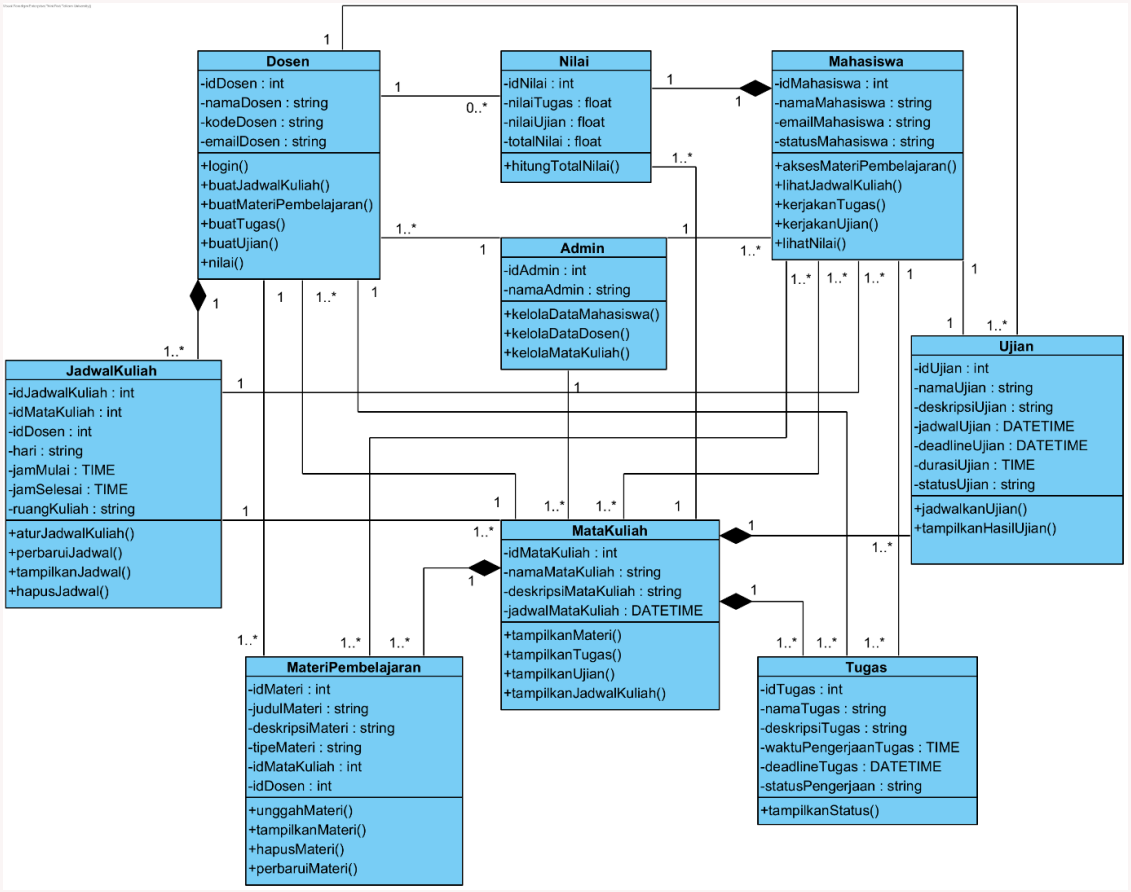
Di sisi pengguna, mahasiswa memulai interaksi dengan platform melalui login ke sistem dengan menggunakan akun yang telah teregistrasi. Setelah berhasil login, mahasiswa akan diarahkan ke halaman dashboard, di mana mahasiswa dapat melihat daftar dan jadwal mata kuliah yang sedang diambil, tugas yang harus dikerjakan, dan jadwal ujian yang akan datang. Setiap mata kuliah yang diambil oleh mahasiswa akan muncul sebagai daftar mata kuliah yang dapat diakses.

Mahasiswa kemudian memilih salah satu mata kuliah yang ingin diakses, dan akan melihat materi yang disediakan oleh dosen. Jika ada tugas yang harus dikerjakan, mahasiswa dapat mengakses dan mengerjakan tugas tersebut. Setelah mengerjakan, mahasiswa mengirimkan tugas yang akan diterima oleh sistem dan diserahkan kepada dosen

Sementara itu, dosen juga melakukan login untuk masuk ke sistem, setelah itu dosen dapat mengakses halaman pengelolaan mata kuliah, yang memungkinkan dosen untuk menetapkan jadwal kuliah, mengunggah materi kuliah, memberikan tugas serta ujian. Dosen juga dapat memantau tugas yang telah dikerjakan oleh mahasiswa dan memberi penilaian.

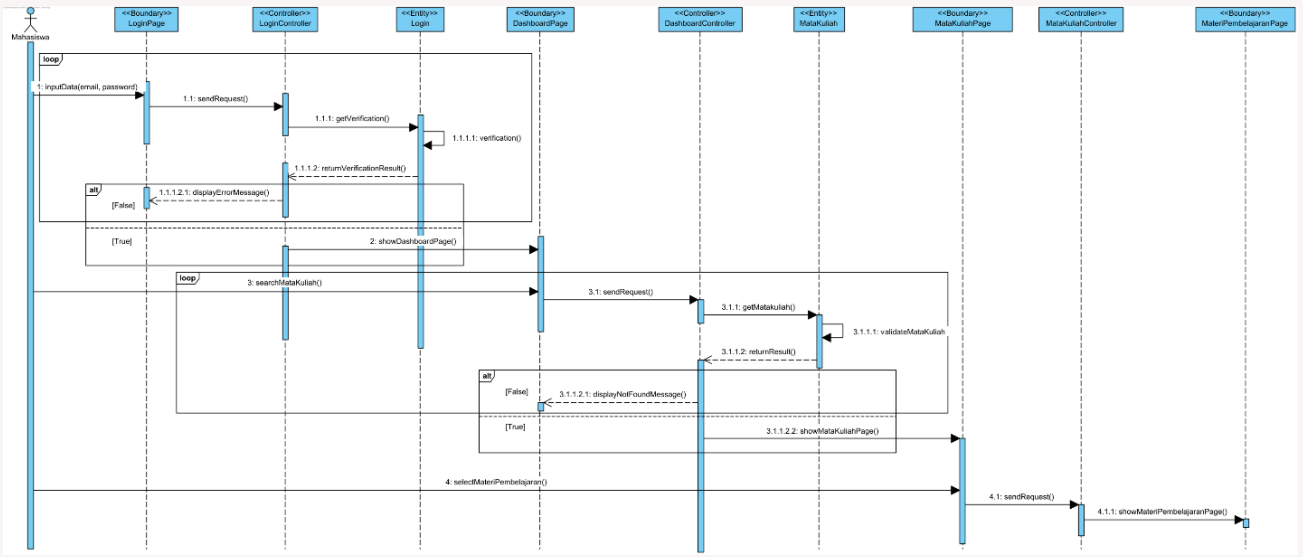
Admin berperan untuk memastikan bahwa seluruh data terintegrasi dengan baik, mulai dari data mahasiswa, dosen, hingga mata kuliah. Admin memiliki kontrol penuh untuk menambah atau menghapus mata kuliah, serta memperbarui data dosen dan mahasiswa. Admin juga memastikan bahwa sistem berjalan dengan lancar.

1. Class Diagram



2. Sequence Diagram

Berikut contoh Sequence Diagram pada fitur Materi Pembelajaran



Good systems design can be a major determinant of business success.

Hendrith Vanlon Smith Jr.